

Today's Topic

Scope resolution operators

Other operators

Type checking

Functions

Arrays

Structure

Union

FUNCTION

- Divide and conquer
 - Construct a program from smaller pieces or components
 - These smaller pieces are called modules
 - Each piece more manageable than the original program
 - Repeated group of instructions in a program is organized as function

Functions in 'c'

- Need of functions :
 - Repetitive Coding can be avoided
 - functional organization of program structure
 - Independent modules : therefore independent coding & debugging is possible.
- Terms in relation with functions :
 - function declaration
 - function definition
 - function call

Functions

- Functions
 - Modularize a program
 - All variables defined inside functions are local variables
 - Known only in function defined
 - Parameters
 - Communicate information between functions
 - Local variables
- Benefits of functions
 - Divide and conquer
 - Manageable program development
 - Software reusability
 - Use existing functions as building blocks for new programs
 - Abstraction - hide internal details (library functions)
 - Avoid code repetition

Function Prototypes

- Function prototype
 - Function name
 - Parameters – what the function takes in
 - Return type – data type function returns (default `int`)
 - Used to validate functions
 - Prototype only needed if function definition comes after use in program
 - The function with the prototype

```
int maximum( int x, int y, int z );
```

 - Takes in 3 ints
 - Return one int

Function Definitions

- Function definition format

```
return-value-type function-name( parameter-list )  
    {  
        declarations and statements  
    }
```

- Function-name: any valid identifier
- Return-value-type: data type of the result (default `int`)
 - `void` – indicates that the function returns nothing
- Parameter-list: comma separated list, declares parameters
 - A type must be listed explicitly for each parameter unless, the parameter is of type `int`

Function Definitions

- Function definition format (continued)

```
return-value-type function-name( parameter-list )  
    {  
        declarations and statements  
    }
```

- Definitions and statements: function body (block)
 - Variables can be defined inside blocks (can be nested)
 - Functions can not be defined inside other functions
- Returning control
 - If nothing returned
 - `return;`
 - or, until reaches right brace
 - If something returned
 - `return expression ;`

Function call/Function overheads /Use of stacks

- A function gets called when function name followed by braces is followed by semicolon
- when a function is called the address of the next instruction is pushed on the stack
- stack is a data structure in memory which is used to execute functions
- when a function is called the control transfer from `main()` to the function definition stored in memory
- when the function execution is over the program control is again return back to `main()` by popping the address in the stack.

Need of return statement

- If we want to return the calculated value from the called function to the calling function then we use return statement.
- Return statement
 - returns calculated value to the calling function
 - returns the control back to the calling function
- a return statement can return only one value at a time
- there may be multiple return statements present in a function but only one of them should be executed depending on the condition.
 - Return value can be a constant, a variable, a user-defined data type, a pointer to a function or a function call.

Pass by value

- There are three argument passing mechanism to functions
 - pass by value
 - pass by address
 - Pass by reference
- the information pass to the function is known as argument
- the arguments that are passed at the time of function call are known as actual arguments
- the arguments that are collected in function definition are known as formal arguments
- actual & formal arguments are stored in different memory locations and they are treated as different variables by the compiler.

Need for pass by address

- By using call by value method only one value can be return
- if multiple values are to be returned then call by address should be used.
- In call by address instead of actual values of arguments , address of the arguments are passed
- hence the manipulations are performed on the actual values of the arguments and not on the formal arguments

Comparison pass by value/ pass by address

- In call by value method photocopy of values of the arguments are passed
- In call by address method address of the arguments are passed
- In call by value method manipulations are on the photocopy values of the arguments.
- In call by address method operations are on the actual of values of the arguments.
- In call by value method only one value is returned explicitly.
- In call by address method more than one value is returned implicitly.

Function Overloading

- While using function overloading you should know
 - Each fn. In C++ is name mangled
 - Name mangling algorithm is different for different compilers eg Microsoft , Borland
 - Therefore C++ compiled code under different compilers may not be compatible
 - Use extern 'C' directive to suppress name mangling

Default Arguments to a Function

- In C++ function can be declared with default values.
- C++ allows to call a function without specifying all it's arguments if a function is declared with default values.
- Only trailing arguments can have default values.
- The default values must be specified in the declaration & not in the definition.

Default Arguments to a Function

✦ Default arguments

```
void f( int x,int y, int z)
{
    cout << x;
    cout << y;
    cout << z;
}
```

```
void main (void)
{
    void f(int,int = 5,int = 0) ;
    f ( ); // error
    f ( 10); //displays10 5 0
    f(10,20,30)// 10 20 30
    f( 10, ,30)//error
}
```

Inline Functions

```
# define    MAX (a, b)    (a > b ? a : b)
```

Problem : preprocessor directives are not typesafe.

Inline Functions

```
int max (int a, int b)
{
    return (a > b ? a : b)
}
```

Problem : Function-call overheads

Inline Functions

```
inline int max (int a, int b)
{
    return (a > b ? a : b)
}
```

Adv. of inline functions : Faster & Typesafe

Difference b/w inline function and Macros

- **Macros have side effects but inline functions do not.**
- **Macros are always expanded but inline functions may or may not. It is the compiler's decision whether to expand the function inline or not.**
- **Macro is not really a function and so usual error checking does not occur during compilation.**
- **Macro is expanded by preprocessor while inline function is parsed by compiler, so better error checking is done by the compiler.**
- **Expressions passed as argument to inline functions are evaluated once. In some cases, expressions passed as arguments to macros can be evaluated more than once.**

Inline Functions

- While using inline function you should know ...
 - inline is a keyword
 - For every instance of a function call the function Definition gets substituted
 - inline is a request to compiler and not instruction
 - Function substitution occurs only at compiler's discretion.

C/C++ Command Line Arguments

- When executing a program in either C or C++ there is a way to pass command line arguments.
- Passed a character arrays.
- Each parameter separated by a space
- Comes into the program as two arguments
 - argc – Number of parameters
 - argv – Parameter list

Command Line Arguments

```
#include <iostream>
using namespace std;

int main(int argc, char*argv[])
{
    for (int i=0; i<argc; i++)
        cout << "This is Argument number #"
            << i
            << "->"
            << argv[i]
            << endl;
    return 0;
}
```

Sample Output

Gaitros>hello Each word should be a unique argument

- This is Argument number #0 -->hello
- This is Argument number #1 -->Each
- This is Argument number #2 -->word
- This is Argument number #3 -->should
- This is Argument number #4 -->be
- This is Argument number #5 -->a
- This is Argument number #6 -->unique
- This is Argument number #7 -->argument

Command Line Arguments

Conventional rules:

- Arguments are always passed to `main( )`.
- There must be two
 - first is an integer
 - second char pointer to an array
- First argument (`argv[0]`) will always be the name of the calling program.
- `argc` will always be at least 1
- The first argument is always `argv[0]`
- The last argument is always `argv[argc-1]`
- `argv[argc]` will always be a null pointer
- Arguments are always passed as character strings. Numbers must be converted from characters to integers, floats, doubles, etc.

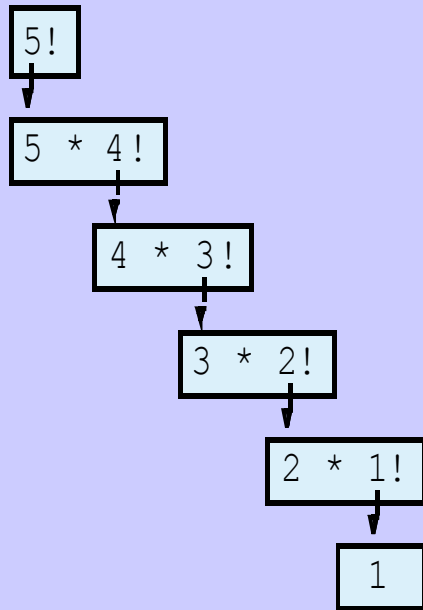
Recursion

- Recursive functions
 - Functions that call themselves
 - Can only solve a base case
 - Eventually base case gets solved
 - Gets plugged in, works its way up and solves whole problem

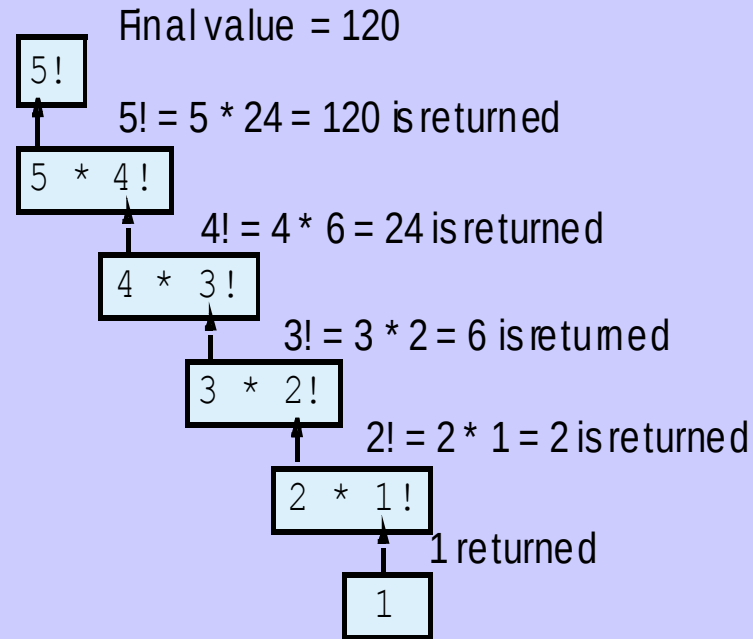
Recursion

- Example: factorials
 - $5! = 5 * 4 * 3 * 2 * 1$
 - Notice that
 - $5! = 5 * 4!$
 - $4! = 4 * 3! \dots$
 - Can compute factorials recursively
 - Solve base case ($1! = 0! = 1$) then plug in
 - $2! = 2 * 1! = 2 * 1 = 2;$
 - $3! = 3 * 2! = 3 * 2 = 6;$

Recursion



(a) Sequence of recursive calls



(b) Values returned from each recursive call.

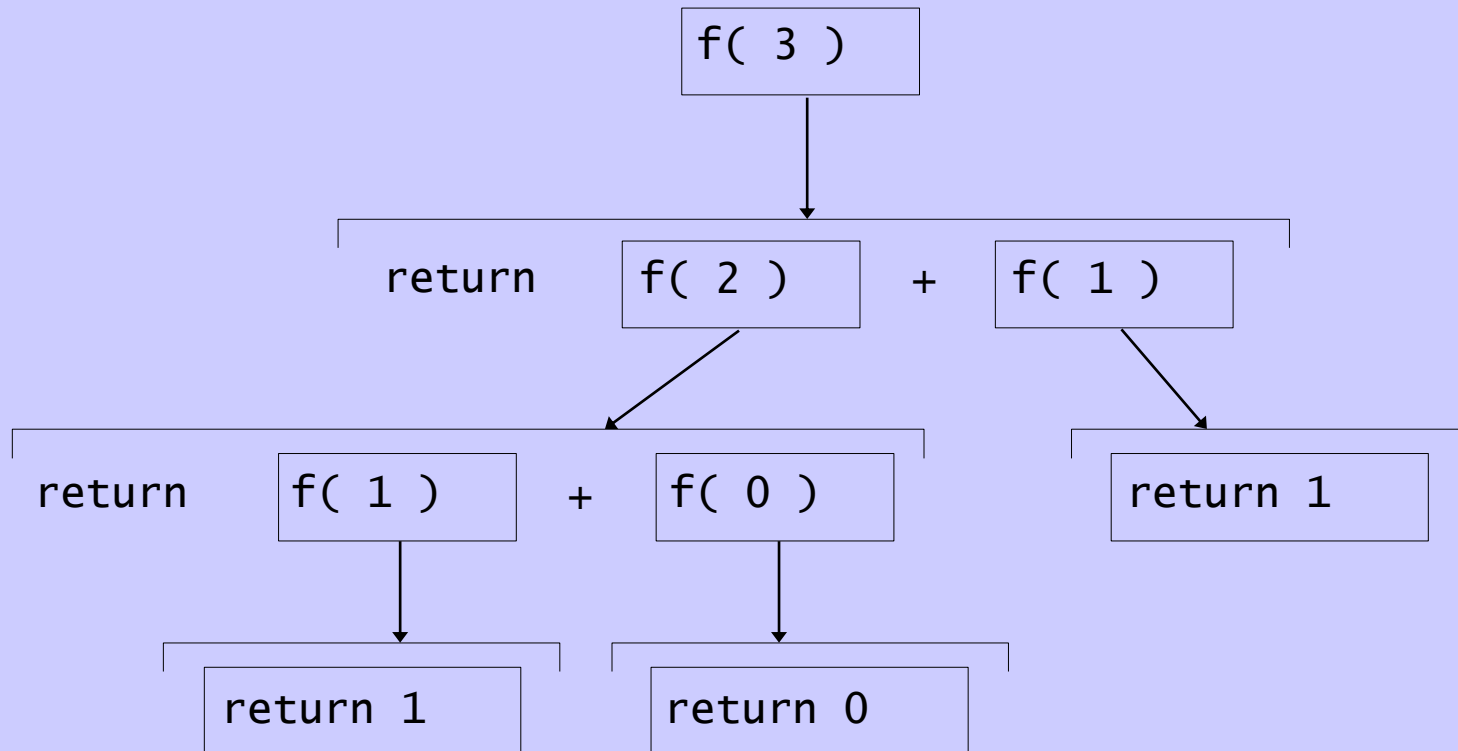
Example Using Recursion: The Fibonacci Series

- Fibonacci series: 0, 1, 1, 2, 3, 5, 8...
 - Each number is the sum of the previous two
 - Can be solved recursively:
 - $\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$
 - Code for the `fibonacci` function

```
long fibonacci( long n )
{
    if (n == 0 || n == 1)  // base case
        return n;
    else
        return fibonacci( n - 1) +
            fibonacci( n - 2 );
}
```

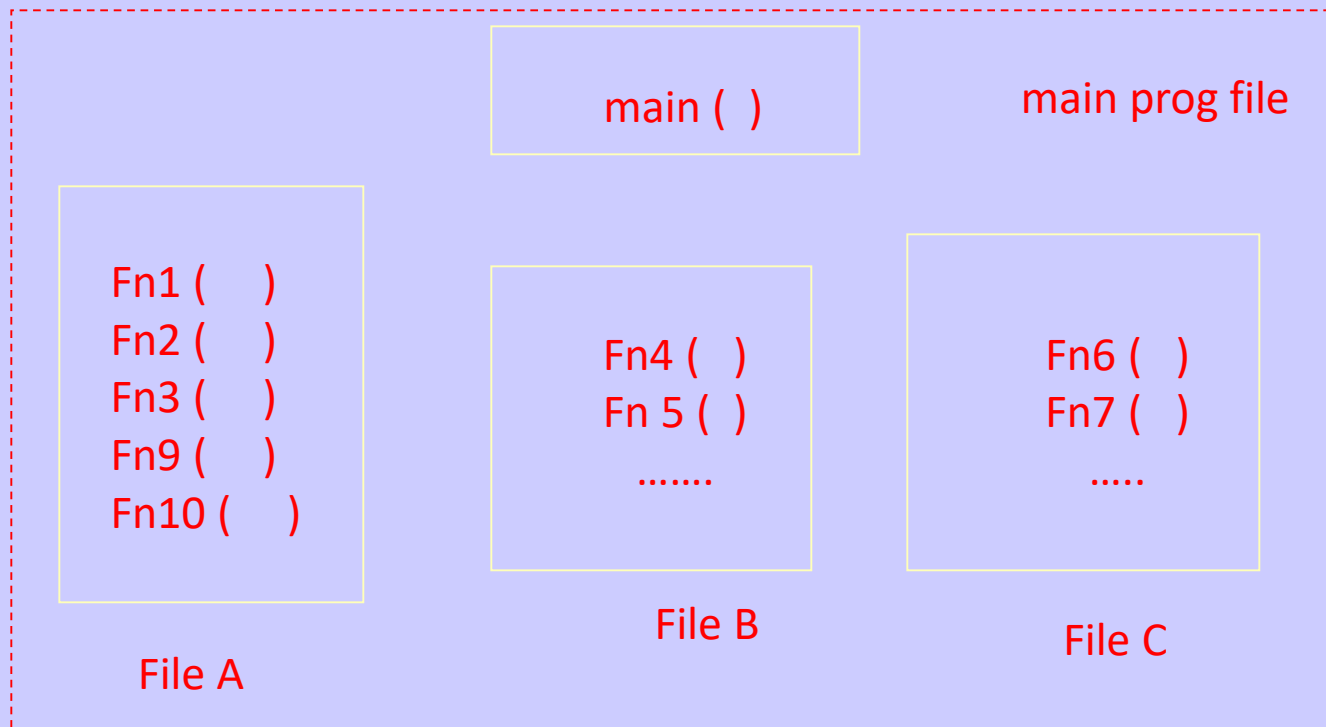
The Fibonacci Series

- Set of recursive calls to function `fibonacci`



Storage class: Concept

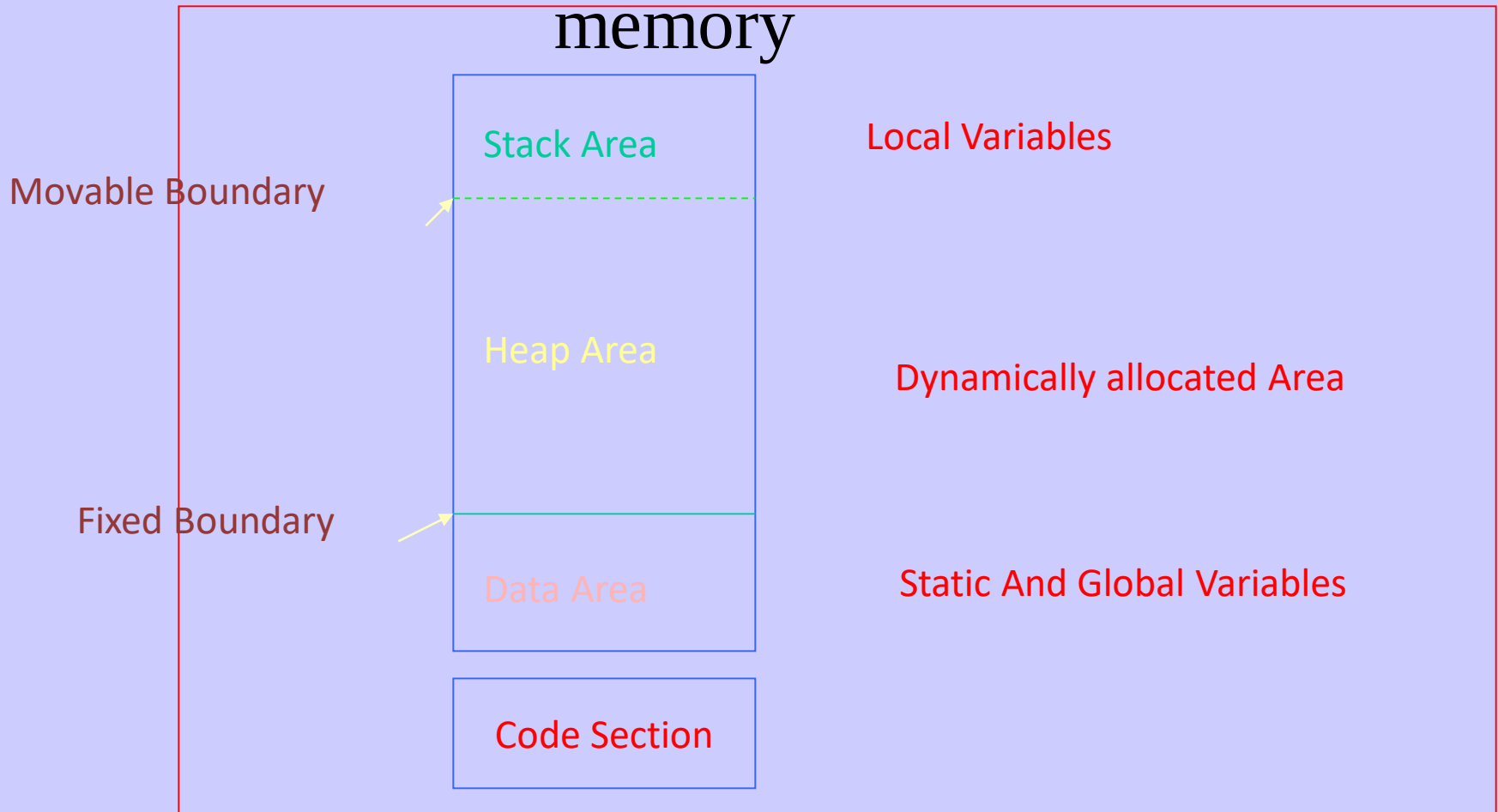
- A Complete Software for an application consists of :



More about Storage Classes

- Variable Declaration consists of
 - ***Storage Class*** Data Type VariableName ;
- four types of storage classes are :
 - auto
 - register
 - static
 - external
- Terms in relation with storage class are :
 - where stored in memory
 - scope / accessibility / visibility
 - life of a variable
 - Definition (only once) & Declaration (can be multiple)

Structure of .exe file in the memory



Complete View : Storage Classes

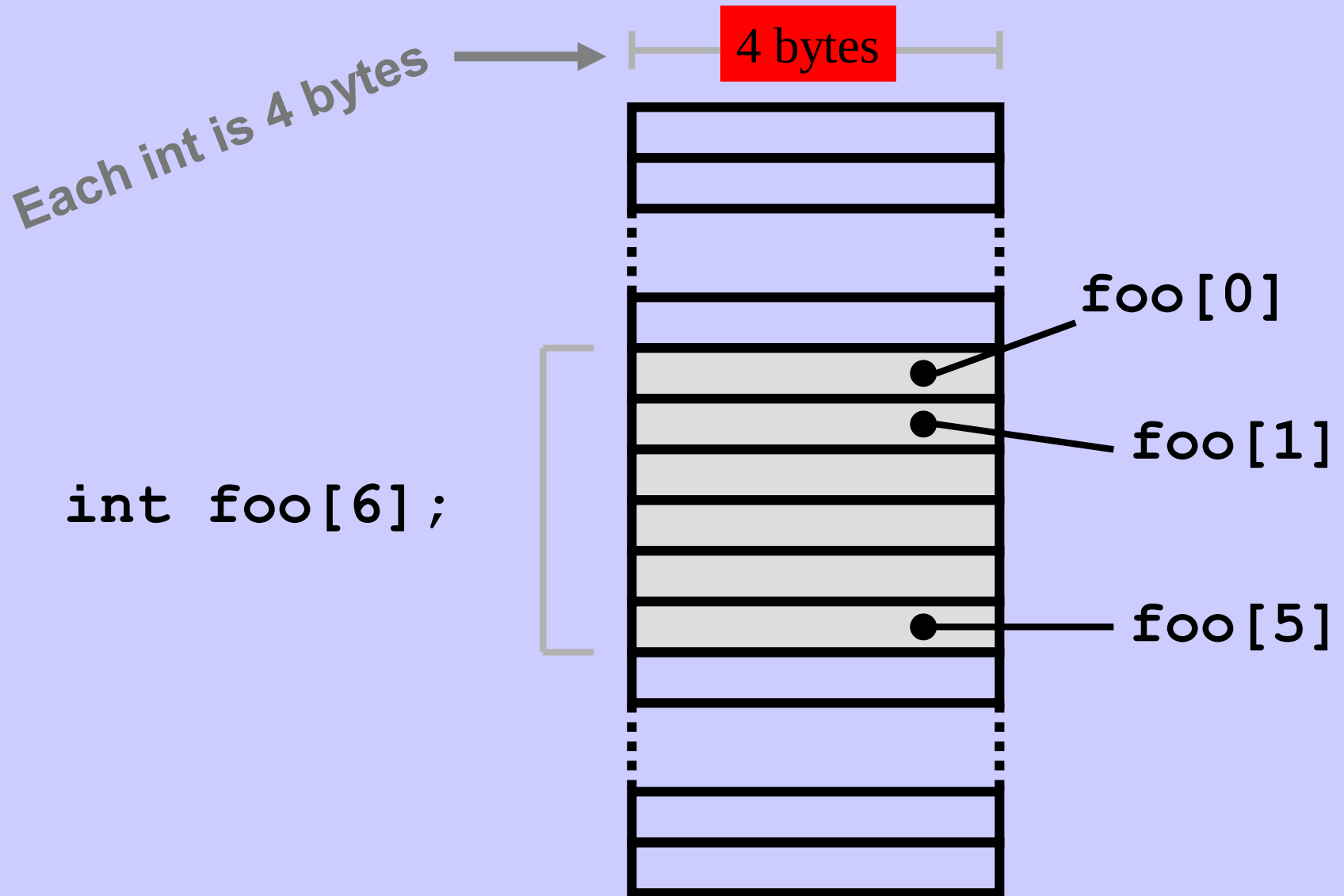
	Auto	Register	Static	Extern
Scope	Local	Local	Within a function	Within a file
Life	Within a fn call	Within a fn call	Through out prg	Through out prg
Initial Value	Garbage	Garbage	Zero	Zero
memory	On stack	Registers	Data Section	Data Section
Declared where	Fn or { }	Fn or { }	Within a function	Define 1 Declare n

C++ Arrays

C++ Arrays

- An array is a consecutive group of memory locations.
- Each *group* is called an element of the array.
- The contents of each element are of the same *type*.
 - Could be an array of int, double, char, ...
- We can refer to individual *elements* by giving the position number (index) of the element in the array.

Memory and Arrays



C++ Arrays start at 0 !!!!!!!

- The first element is the 0th element!
- If you declare an array of n elements, the last one is number $n-1$.
- If you try to access element number n it is an error!

If only millenniums started at 0 ...

Array Subscripts

- The element numbers are called subscripts.

Array name → **foo[i]** ← *subscript*

A subscript can be any integer expression:

These are all valid subscripts:

foo[17] **foo[i+3]** **foo[a+b+c]**

Array Example

```
int main(void) {  
    int facs[10];  
  
    for (int i=0;i<10;i++)  
        facs[i] = factorial(i);  
  
    for (int i=0;i<10;i++)  
        cout << "factorial(" << i << ") is " <<  
            facs[i] << endl;  
}
```

Declaring An Array

```
element_type array_name[number_of_elements];
```

`element_type` can be any C++ variable type.

`array_name` can be any valid variable name.

`number_of_elements` can be an expression.

Initialization

- You can initialize an array when you declare it (just like with variables):

```
int foo[5] = { 1,8,3,6,12};
```

```
double d[2] = { 0.707, 0.707};
```

```
char s[] = { 'R', 'P', 'I' };
```



You don't need to specify a size when initializing, the compiler will count for you.

An array printing function

Can pass an array as a parameter.
You don't have to say how big it is!



```
void print_array(int a[], int len) {  
    for (int i=0;i<len;i++)  
        cout << "[" << i << "]" = "  
            << a[i] << endl;  
}
```

What if we want to print doubles?

- The `print_array` function is declared to handle only `ints`.
- We can write another function that can be used to print doubles.
- We have to write another function (we can't use the same one).
 - Not really true – this is what templates can do for you!

`print_array()` for doubles

```
void print_array(double a[], int len) {  
    for (int i=0;i<len;i++)  
        cout << "[" << i << "] = "  
            << a[i] << endl;  
}
```

Which is it?

- We now have two functions with the same name:

```
void print_array(double a[], int len);  
void print_array(int a[], int len);
```

This is fine – as long as the prototypes are different everything works.

This is called "overloading", using the same name for two (or more) different functions.

Arrays of `char` are special

- C++ provides a special way to deal with arrays of characters:

```
char string1[] =  
    "RPI without PI is like meat without eat";
```

- `char` arrays can be initialized with string literals.

Arrays of Arrays

- You can create an array of arrays:

```
int a[2][2];
```

```
for (int i=0;i<2;i++)  
    for (int j=0;j<2;j++)  
        a[i][j] = i+j;
```

2-D Array: **int A[3][4]**

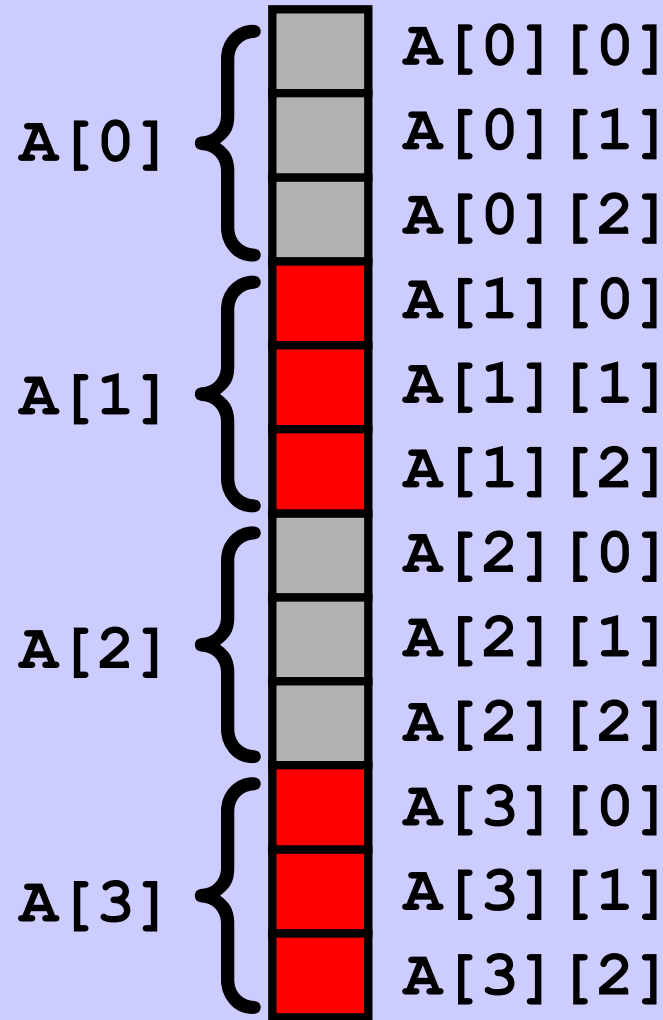
	Col 0	Col 1	Col 2	Col 3
Row 0	A[0][0]	A[0][1]	A[0][2]	A[0][3]
Row 1	A[1][0]	A[1][1]	A[1][2]	A[1][3]
Row 2	A[2][0]	A[2][1]	A[2][2]	A[2][3]

2-D Memory Organization

```
char A[4][3];
```

A is an array of size 4.

Each element of A is
an array of 3 chars



2-D Array Example

```
const int NumStudents = 10;
const int NumHW = 3;
double grades[NumStudents][NumHW];

for (int i=0;i<NumStudents;i++) {
    for (int j=0;j<NumHW;j++) {
        cout << "Enter HW " << j <<
            " Grade for student number " <<
            i << endl;
        cin >> grades[i][j];
    }
}
```

2-D Array (cont.)

You don't need to specify the size of the first dimension
You must include all other sizes!



```
double student_average( double g[][NumHW], int stu) {  
    double sum = 0.0;  
    for (int i=0;i<NumHW;i++)  
        sum += g[stu][i];  
  
    return (sum/NumHW) ;  
}
```

Another way

```
double array_average( double a[], int len) {  
    double sum = 0.0;  
    for (int i=0;i<len;i++)  
        sum += a[i];  
  
    if (len==0)  
        return(0);  
    else  
        return(sum/len);  
}
```

Division by 0 is bad idea!



Arrays and pass-by-reference

- Arrays are always passed by reference
 - Inside a function any changes you make to array values are for keeps!
 - You can write functions that modify the contents of an array.
 - You need to make sure that a function knows how big the array is!!!

A Bad Idea

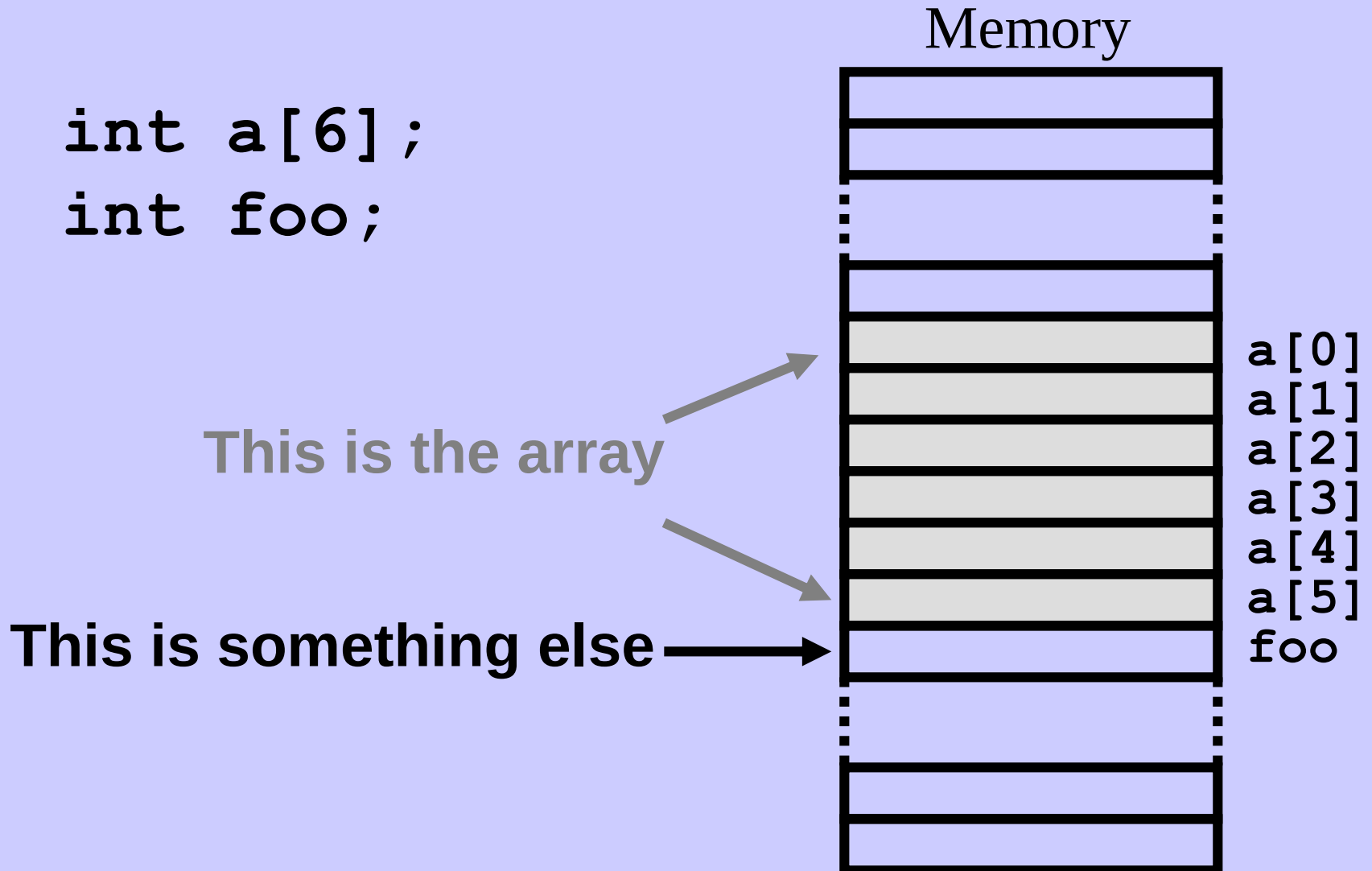
```
int read_array( int a[] ) {  
    int i=0;  
    int val;  
    do {  
        cout << "Enter next value, 0 to end\n";  
        cin >> val;  
        if (val) a[i++] = val;  
    } while (val);  
    return(i); // returns the number or numbers  
}
```

The problem is that the function might go beyond the size of the array.

Result: Segmentation Violation (or worse!).

C++ does not have *bounds checking*

```
int a[6];  
int foo;
```



1 D Character Array ->String

- Strings are nothing but a single dimensional character array.
- Strings i.e character arrays are used for text processing.
- Declaration for character array is `char ArrName[size];`
- Every string ends with a ‘\0’ character whose ASCII value is zero .
- Null character is automatically inserted by the compiler at the end of string.
- %s is the format specifier used for strings

Arrays and Strings...

- Operations performed on character strings include:
 - Reading and writing strings
 - Copying one string to another
 - Combining strings together
 - Comparing strings for equality
 - Extracting a portion of a string

- A string variable is declared as an array of characters.
- Syntax:
`char string_name[size];`
- E.g. `char name[20];`
- When the compiler assigns a character string to a character array, it automatically supplies a *null character* (`'\0'`) at the end of the string

Initializing String Variables

- Strings are initialized in either of the following two forms:

```
char name[4]={‘R’,‘A’,‘M’, ‘\0’};
```

```
char name[]={‘R’,‘A’,‘M’, ‘\0’};
```

```
char name[4] = "RAM";
```

```
char name[] = "RAM";
```

R	A	M	\0
name[0]	name[1]	name[2]	name[3]

- When we initialize a character array by listing its elements, the null terminator or the size of the array must be provided explicitly.

Standard library function for strings

- all standard string functions are in header file <string.h>
- commonly used standard string functions
 - strlen(stringname):- calculates string length
 - strcpy(stringname,stringname):- copies one string into another
 - strcmp(stringname,stringname):-compare one string with another
 - returns 0 if strings are similar
 - returns >0 if first is greater than another
 - returns <0 if first is less than another
 - strcat(stringname,stringname):-concates one string at the end of another

STRUCTURE

Need of structure

- Array can store identical datatype elements
- but reality demands something else
 - e.g. student
 - student name(char array), student roll_no(int), student marks(float) etc.
- Hence a datatype is required which can store dissimilar data type.
- Structure is such a built in secondary datatype.

Structure Data Type

A structure is a user defined data type that groups logically related data items of different data types into a single unit. All the elements of a structure are stored at contiguous memory locations. A variable of structure type can store multiple data items of different data types under the one name. As the data of employee in company that is name, Employee ID, salary, address, phone number is stored in structure data type.

Defining of Structure

A structure has to be defined, before it can be used. The syntax of defining a structure is

```
struct <struct_name>
```

```
{
```

```
<data_type> <variable_name>;
```

```
<data_type> <variable_name>;
```

```
.....
```

```
<data_type> <variable_name>;
```

```
};
```


Example of Structure

The structure of Employee is declared as

```
struct    employee
{
int        emp_id;
char       name[20];
float      salary;
char       address[50];
int        dept_no;
int        age;
};
```

Memory Space Allocation

8000	emp_id	employee
8002	name[20]	
8022		
8024	salary	
	address[50]	
8074		
8076		
8078		
	dept_no	
	age	

Declaring a Structure Variable

A structure has to be declared, after the body of structure has been defined. The syntax of declaring a structure is

Struct <struct_name> <variable_name>;

The example to declare the variable for the defined structure “employee”

```
struct employee e1;
```

Here e1 variable contains 6 members that are defined in structure.

Initializing a Structure Members

The members of individual structure variable is initialize one by one or in a single statement. The example to initialize a structure variable is

```
1) struct employee e1 = {1, "Hemant",12000, "3
    vikas colony new delhi",10, 35);
```

```
2) e1.emp_id=1;           e1.dept_no=1
   e1.name="Hemant";      e1.age=35;
   e1.salary=12000;
   e1.address=" LPU JALANDHAR";
```

Accessing a Structure Members

The structure members cannot be directly accessed in the expression. They are accessed by using the name of structure variable followed by a dot and then the name of member variable. The method used to access the structure variables are `e1.emp_id`, `e1.name`, `e1.salary`, `e1.address`, `e1.dept_no`, `e1.age`. The data with in the structure is stored and printed by this method using `cin` and `cout` statement in `c++` program.

Structure Assignment

The value of one structure variable is assigned to another variable of same type using assignment statement. If the e1 and e2 are structure variables of type employee then the statement

$$e1 = e2;$$

assign value of structure variable e2 to e1. The value of each member of e2 is assigned to corresponding members of e1.

Access the Pointer in Structures

The member of structure variable is accessed by using the pointer variable with arrow operator($\rightarrow$) instead of period operator(.). The syntax to access the pointer to structure.

`pointer_var_name $\rightarrow$ structure_member;`

For Example:

`emp $\rightarrow$ name;`

Here “name” structure member is accessed through pointer variable emp.

Union Data Type

A union is a user defined data type like structure. The union groups logically related variables into a single unit. The union data type allocate the space equal to space need to hold the largest data member of union. The union allows different types of variable to share same space in memory. There is no other difference between structure and union than internal difference. The method to declare, use and access the union is same as structure.

Defining of Union

A union has to be defined, before it can be used. The syntax of defining a structure is

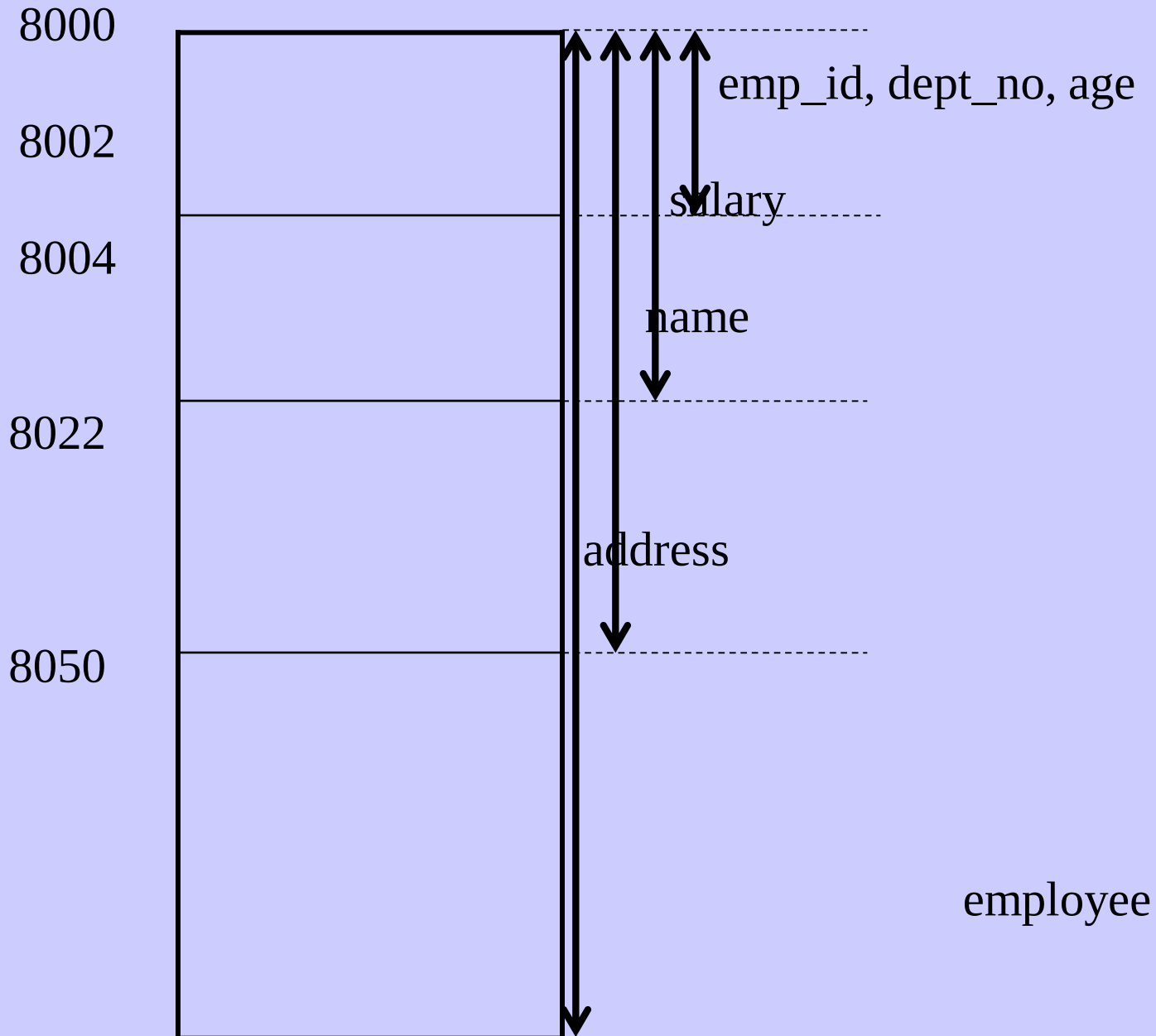
```
union <union_name>
{
    <data_type> <variable_name>;
    <data_type> <variable_name>;
    .....
    <data_type> <variable_name>;
};
```

Example of Union

The union of Employee is declared as

```
union employee
{
int emp_id;
char name[20];
float salary;
char address[50];
int dept_no;
int age;
};
```

Memory Space Allocation



Difference between Structures & Union

- 1) The memory occupied by structure variable is the sum of sizes of all the members but memory occupied by union variable is equal to space hold by the largest data member of a union.
- 2) In the structure all the members are accessed at any point of time but in union only one of union member can be accessed at any given time.

Summary

- A structure is a user defined data type that groups logically related data items of different data types into a single unit.
- The elements of a structure are stored at contiguous memory locations.
- The value of one structure variable is assigned to another variable of same type using assignment statement.
- An array of variables of structure is created.
- A variable of structure type is defined as a member of other structure type called nested structure.

Summary

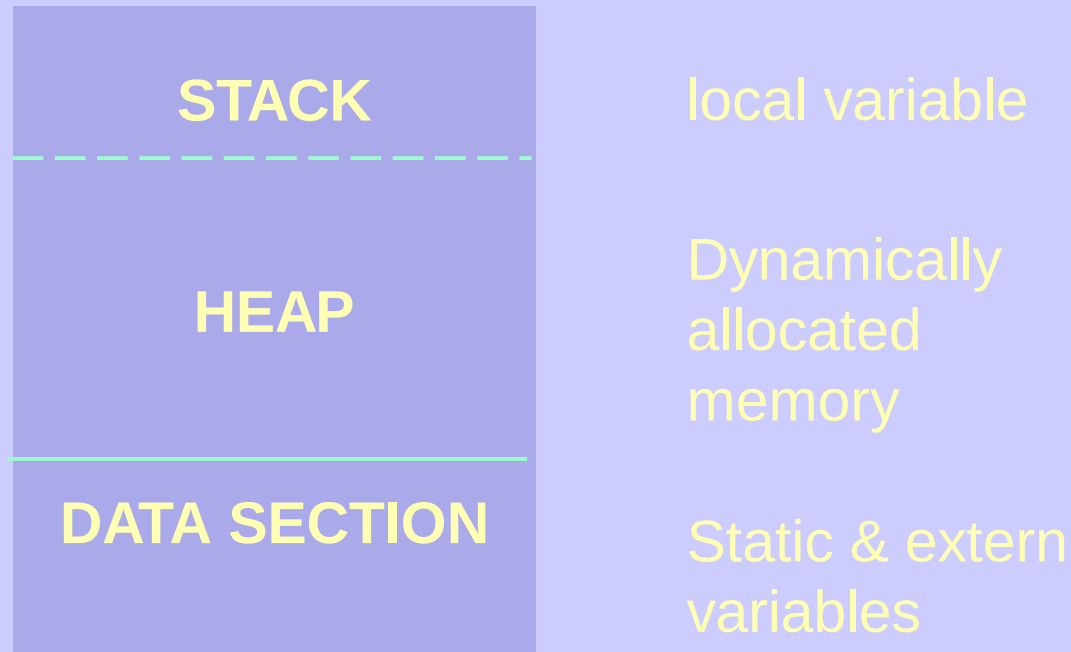
- The member of structure variable is accessed by pointer variable with arrow operator ($\rightarrow$).
- The structure variable can be passed to a function as a parameter.
- The function can return a variable of structure type.
- A union is like structure that group logically related variables into a single unit. The union allocate the space equal to space need to hold the largest data member of union.
- Structure used in database management and many more applications.

Dynamic memory allocation

- to allocate memory on heap dynamically
use :
- malloc()
 - standard function used for dynamic memory allocation
- free()
 - A function used to release dynamically allocated memory.

Dynamic Memory Allocation

- Structure of a data segment of .exe file in memory



Dynamic Memory Allocation

- new operator : allocates memory on heap
delete operator frees allocated memory by new

```
{  
    char * str ;  
    str = new char [100] ;  
    cin >> str ;  
    cout << str ;  
    delete[] str ;  
}
```

Dynamic Memory Allocation

- While using new/delete you should know ...
 - No need of sizeof operator
 - new returns pointer of specified data type
 - Avoid mix use of malloc & new
 - if [] is used in new, use [] in delete also.

Otherwise there will be problem of memory leakage

Dynamic Memory Allocation

Memory Leakage

```
int *p = new int [100];
```

```
.....;
```

```
.....;
```

```
delete [] p;
```

4 bytes

400
bytes

if delete p

Only first 4 bytes are freed

which results in memory leakage of

396 bytes

Pointers in C++

Pointers

- Often referred to as *references*
- A pointer contains the location, or *address* in memory, of a memory cell
 - Initially undefined
 - A statically allocated pointer declaration
`int *p;`
 - A dynamically allocated `int` for `p` to point at
`p = new int;`

Dereferencing Operator

- The expression `*p` denotes the memory cell to which `p` points
- Here, `*` is called the *dereferencing operator*
- Be careful not to dereference a pointer that has not yet been initialized:

```
int *p;
```

p

?

Address in `p` could be any memory location

```
*p = 7;
```

Attempt to put a value into an unknown memory location will result in a *run-time error*, or worse, a *logic error*

Dereferencing Operator

- To help keep things straight, it's useful to ***pretend*** there are parentheses in the pointer declarations:
 - `(int *) p`; means that `p` is of type `int*` (i.e.: that is, `p` is a pointer to an `int` location)
- *Don't actually write declarations this way*

The Address Operator

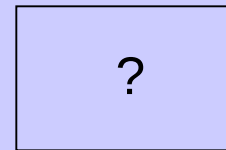
- The expression `&x` denotes the address of a variable `x`
- Here, `&` is called the *address operator*

```
int x, *p;
```

x

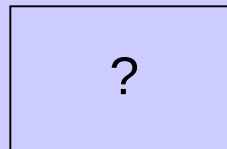


p

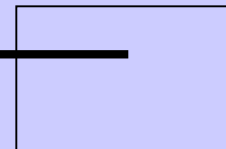


```
p = &x;
```

x



p

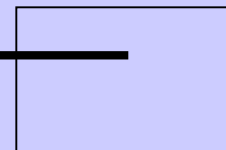


```
*p = 4;
```

x



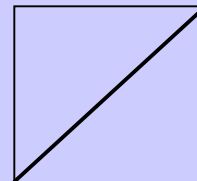
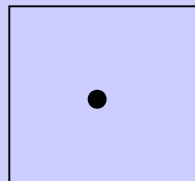
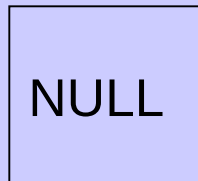
p



Value of `x` has
been changed
by
`*p = 4;`

The Null Pointer

- The *null pointer* is a special constant which is used to explicitly indicate that a pointer does not point anywhere
 - **NULL** is defined in the standard library `<cstdlib>` (or `<stdlib.h>`)
 - In diagrams, indicated as one of:

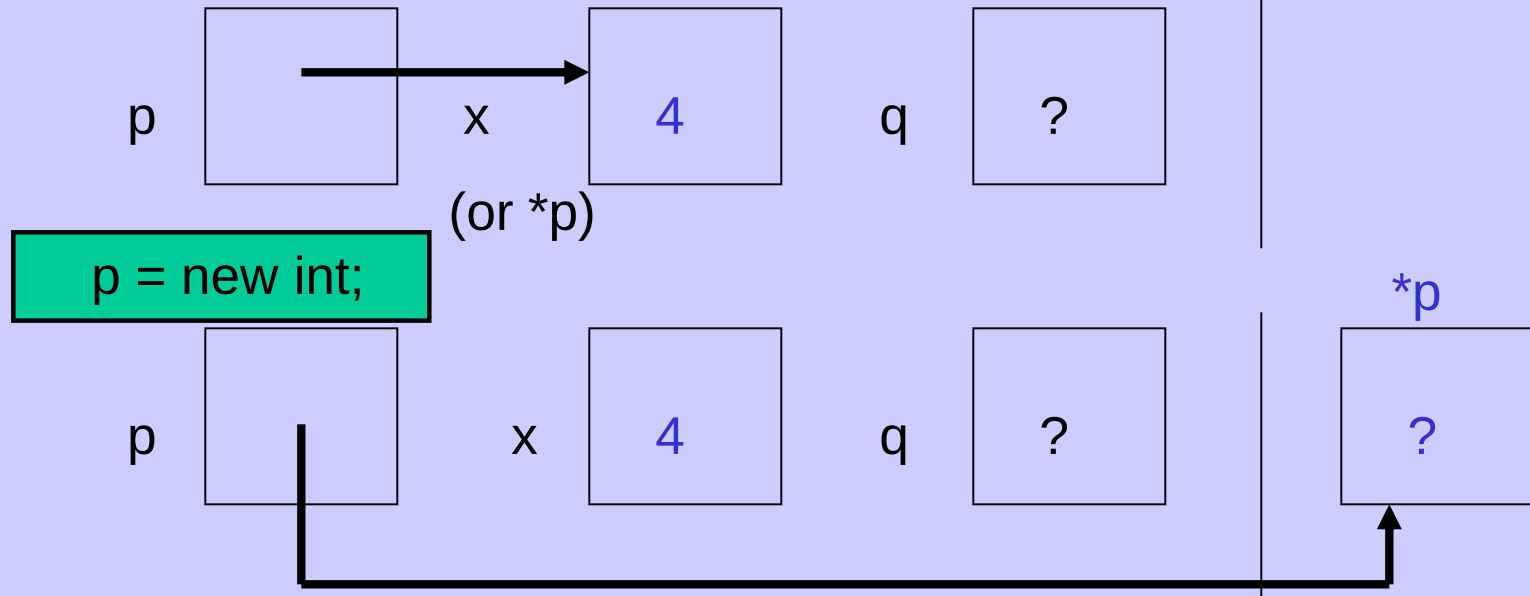


Pointer Example

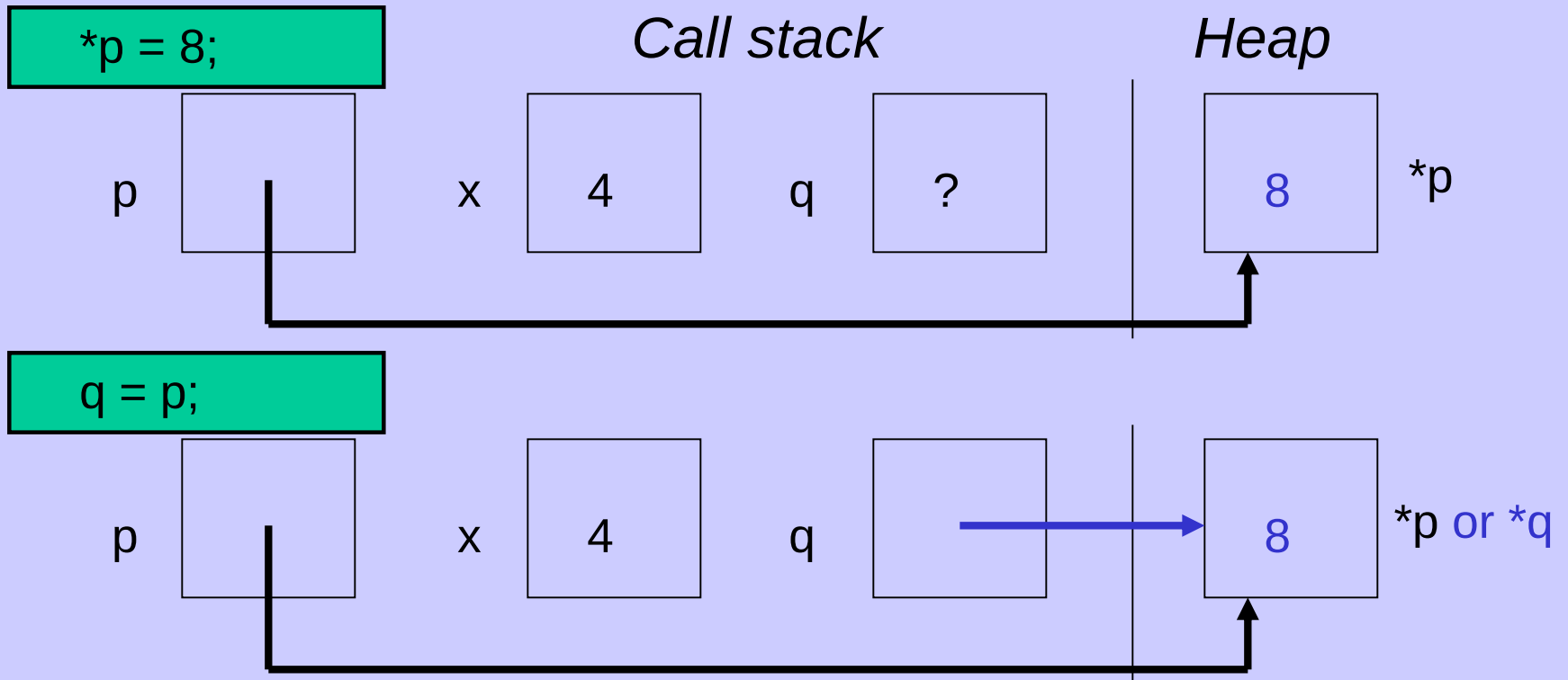
```
int *p, *q, x;  
p = &x;  
*p = 4;
```

Call stack

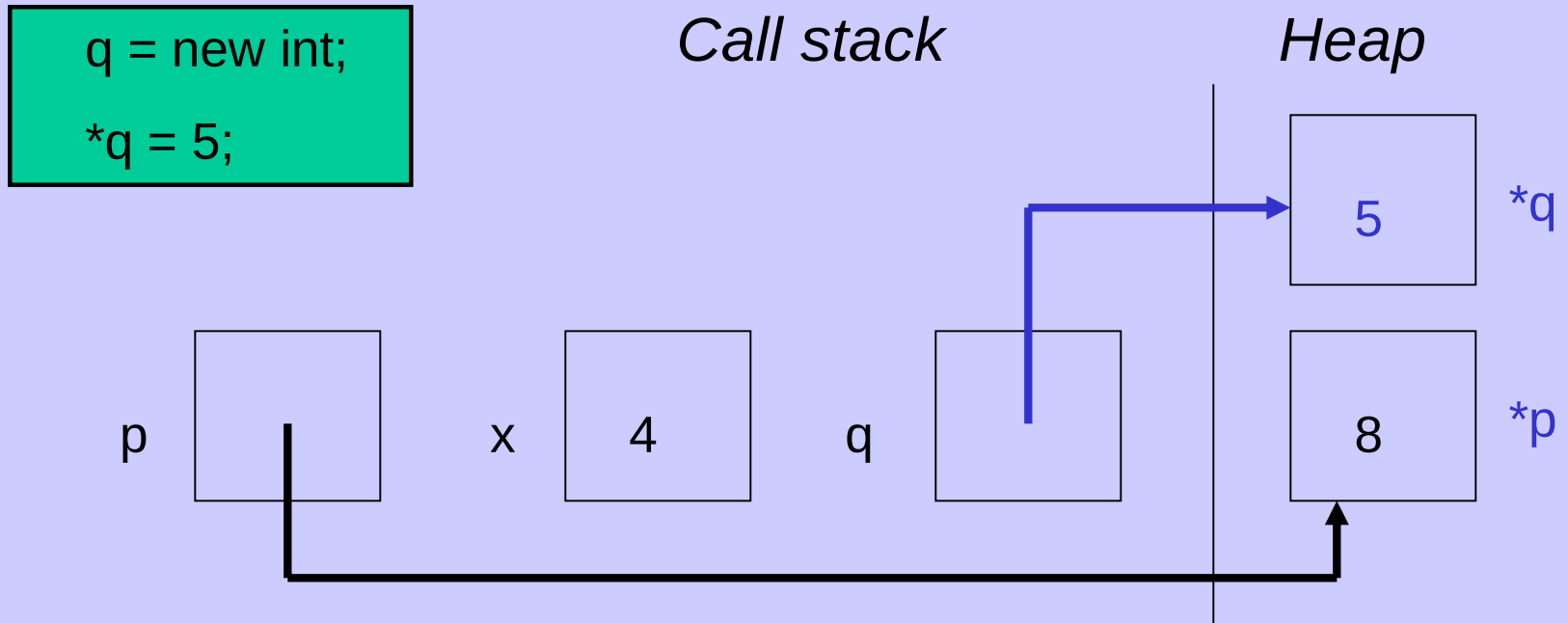
Heap



Pointer Example (cont'd)



Pointer Example (cont'd)

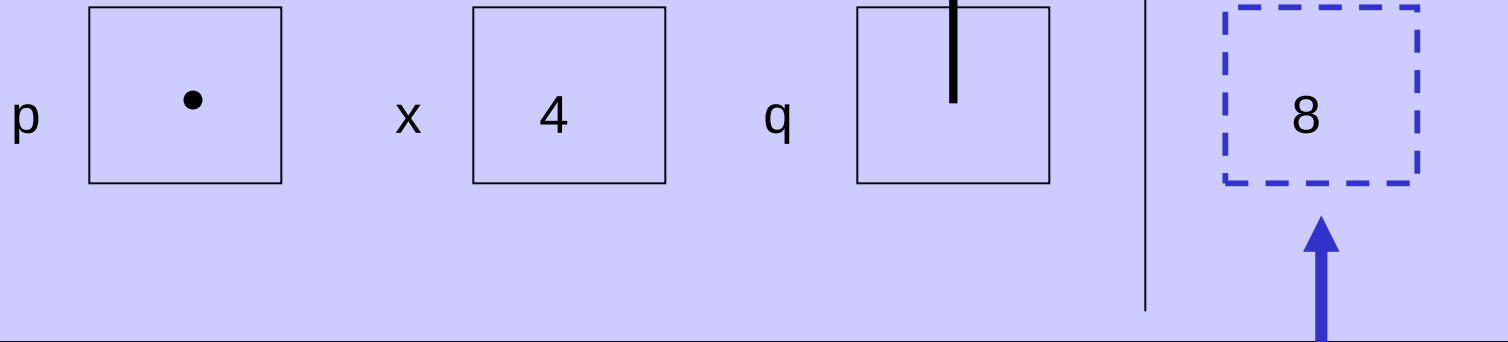


Pointer Example (cont'd)

`p = NULL;`

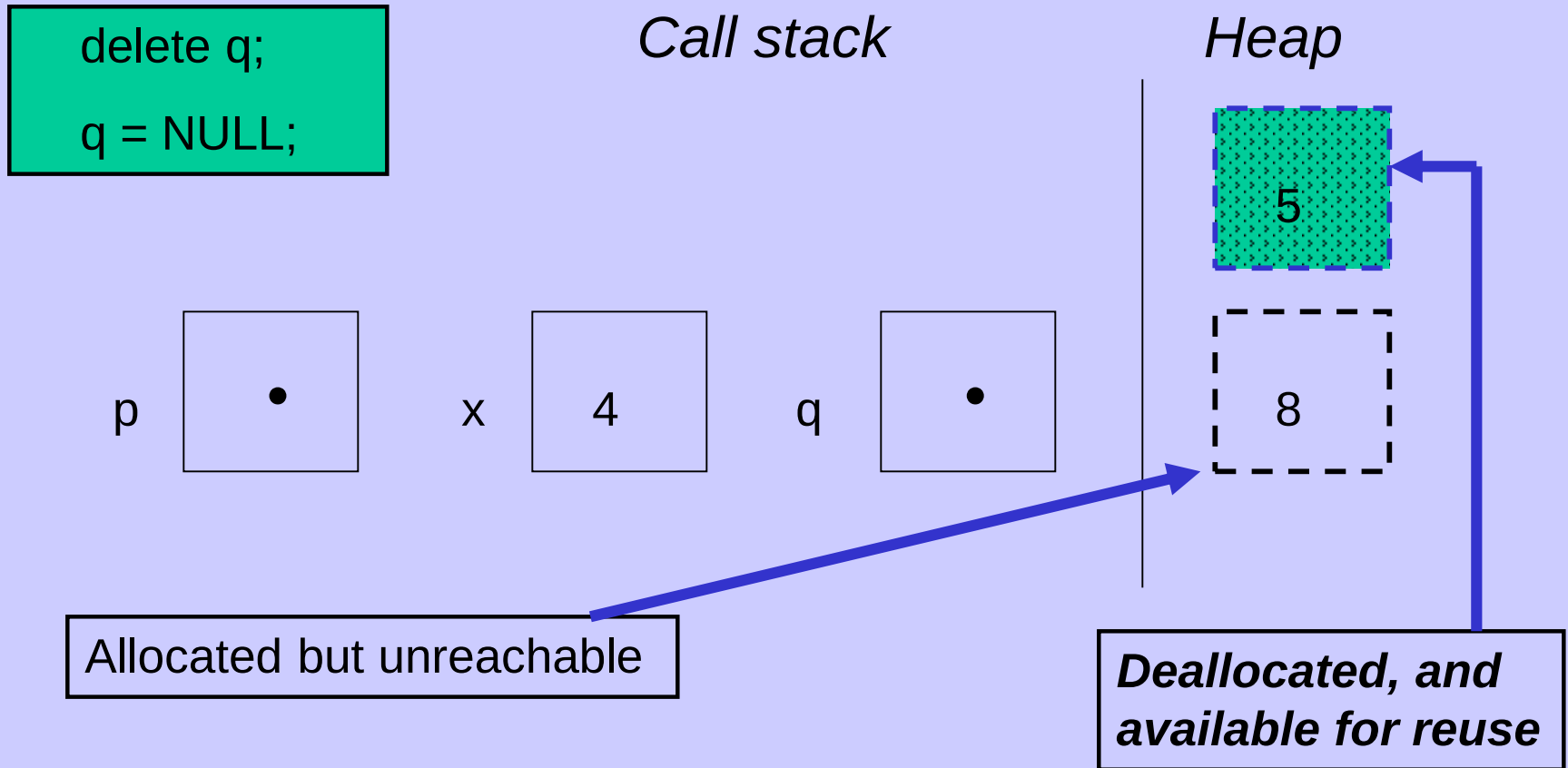
Call stack

Heap



This location is no longer has a name, is no longer reachable from the program, and cannot be deallocated: we have a **memory leak**

Pointer Example (cont'd)



The Basics

- A pointer is merely an address of where a datum or structure is stored
 - all pointers are typed based on the type of entity that they point to
 - to declare a pointer, use `*` preceding the variable name as in `int *x;`
- To set a pointer to a variable's address use `&` before the variable as in `x = &y;`
 - `&` means “return the memory address of”
 - in this example, `x` will now point to `y`, that is, `x` stores `y`'s address
- If you access `x`, you merely get the address
- To get the value that `x` points to, use `*` as in `*x`
 - `*x = *x + 1;` will add 1 to `y`
- `*` is known as the *indirection* (or dereferencing) operator because it requires a second access
 - that is, this is a form of indirect addressing

Example Code

```
int x = 1, y = 2, z[10];
int *ip;           // ip is a pointer to an int, so it can point to x, y, or an element of z

ip = &x;           // ip now points at the location where x is stored
y = *ip;           // set y equal to the value pointed to by ip, or y = x
*ip = 0;           // now change the value that ip points to to 0, so now x = 0
                  // but notice that y is unchanged
ip = &z[0];        // now ip points at the first location in the array z

*ip = *ip + 1;     // the value that ip points to (z[0]) is incremented
```

```
int x, *y, z, *q;
x = 3;
y = &x;            // y points to x
printf("%d\n", x);  // outputs 3
printf("%d\n", y);  // outputs x's address, will seem like a random number to us
printf("%d\n", *y); // outputs what y points to, or x (3)
printf("%d\n", *y+1); // outputs 4 (print out what y points to + 1)
printf("%d\n", *(y+1)); // this outputs the item after x in memory – what is it?
z = *(&x);          // z equals 3 (what &x points to, which is x)
q = &*y;            // q points to 3 – note *& and &* cancel out
```


Arrays and Pointers

- We declare an array using [] in our declaration following the variable name
 - `int x[5];` // unlike Java, we can't do `int[ ] x;`
- You must include the size of the array in the [] when declaring unless you are also initializing the array to its starting values as in:
 - `int x [ ] = {1, 2, 3, 4, 5};`
 - you can also include the size when initializing as long as the size is $\geq$ the number of items being initialized (in which case the remaining array elements are uninitialized)
- As in Java
 - you access array elements just as in Java as in `x[4]`
 - array indices start at 0
 - arrays can be passed as parameters, the type being received would be denoted as `int x[ ]`
- Arrays in C are interesting because they are pointed to
 - the variable that you declare for the array is actually a pointer to the first array element
- You can interact with the array elements either through pointers or by using []
- One of the intriguing features of pointers in C is the ability to manipulate the pointers through pointer arithmetic – a pointer is an int value, so we can add or subtract
 - this will be used for stepping through arrays rather than using array indices

Using Pointers with Arrays

- Recall in an earlier example, we did `ip = &z[0];`
- This sets our pointer to point at the first element of the array
 - In fact, `z` is a pointer as well and we can access `z[0]` either using `z[0]`, `*ip`, or `*z`
- What about accessing `z[1]`?
 - We can do `z[1]` as usual, or we can add 1 to the location pointed to by `ip` or `z`, that is `*(ip+1)` or `*(z+1)`
 - While we can reset `ip` to be `ip = ip+1`, we cannot reset `z` to be `z = z+1`
 - adding 1 to `ip` will point to `z[1]`, but if `z = z + 1` were legal, we would lose access to the first array location since `z` is our array variable
- Notice that `ip=ip+1` (or `ip++`) moves the pointer 4 bytes instead of 1 to point at the next array location
 - The amount added to the pointer is based on the size of the array element
 - 8 for an array of doubles
 - 1 for an array of chars (strings)
 - 4 for an array of ints
- We can declare our arrays using pointers instead of `[ ]`
 - notably, we might do this for our formal parameters as this better describes what we are dealing with)
 - for instance, `function1(int *array)` rather than `function1(int[ ] array)`
 - We wouldn't normally do this to declare our arrays as the array's pointer exists but not the array itself

Iterating Through the Array

- Here we see two ways to iterate through an array, the usual way, but also a method using pointer arithmetic

```
int j;  
for(j = 0; j < n; j++)  
    a[j]++;
```

```
int *pj;  
for(pj = a; pj < a + n; pj++)  
    (*pj)++;
```

- Let's consider the code on the right:
 - pj is a pointer to an int
 - We start with pj pointing at a, that is, pj points to a[0]
 - The loop iterates while $pj < a + n$
 - pj is a pointer, so it is an address
 - a is a pointer to the beginning of an array of n elements so $a + n$ is the size of the array
 - $pj++$ increments the pointer to point at the next element in the array
 - The instruction $(*pj)++$ says “take what pj points to and increment it”
 - NOTE: $(*pj)++$; increments what pj points to, $*(pj++)$; increments the pointer to point at the next array element
 - what do each of these do? $*pj++$; $++*pj$;

Array Example Using a Pointer

```
int x[4] = {12, 20, 39, 43}, *y;
y = &x[0];                // y points to the beginning of the array
printf("%d\n", x[0]);      // outputs 12
printf("%d\n", *y);        // also outputs 12
printf("%d\n", *y+1);      // outputs 13 (12 + 1)
printf("%d\n", (*y)+1);    // also outputs 13
printf("%d\n", *(y+1));    // outputs x[1] or 20
y+=2;                     // y now points to x[2]
printf("%d\n", *y);        // prints out 39
*y = 38;                  // changes x[2] to 38
printf("%d\n", *y-1);      // prints out x[2] - 1 or 37
*y++;                     // sets y to point at the next array element
printf("%d\n", *y);        // outputs x[3] (43)
(*y)++;                   // sets what y points to to be 1 greater
printf("%d\n", *y);        // outputs the new value of x[3] (44)
```

Strings

- There is no string type, we implement strings as arrays of chars
 - `char str[10];` // str is an array of 10 chars or a string
 - `char *str;` // str points to the beginning of a string of unspecified length
- There is a `string.h` library with numerous string functions
 - they all operate on arrays of chars and include:
 - `strcpy(s1, s2)` – copies s2 into s1 (including ‘\0’ as last char)
 - `strncpy(s1, s2, n)` – same but only copies up to n chars of s2
 - `strcmp(s1, s2)` – returns a negative int if $s1 < s2$, 0 if $s1 = s2$ and a positive int if $s1 > s2$
 - `strncmp(s1, s2, n)` – same but only compares up to n chars
 - `strcat(s1, s2)` – concatenates s2 onto s1 (this changes s1, but not s2)
 - `strncat(s1, s2, n)` – same but only concatenates up to n chars
 - `strlen(s1)` – returns the integer length of s1
 - `strchr(s1, ch)` – return a pointer to the first occurrence of ch in s1 (or NULL if ch is not present)
 - `strrchr(s1, ch)` – same but the pointer points to the last occurrence of ch
 - `strpbrk(s1, s2)` – return a pointer to the first occurrence of any character in s1 that matches a character in s2 (or NULL if none are present)
 - `strstr(s1, s2)` – substring, return a pointer to the char in s1 that starts a substring that matches s2, or NULL if the substring is not present

Implementing Some of These

```
int strlen(char *s)
{
    int n;
    for(n = 0; *s != '\0'; s++)
        n++;
    return n;
}
```

```
int strcmp(char *s, char *t)
{
    int i;
    for(i=0; s[i] == t[i]; i++)
        if(s[i] == '\0')
            return 0;
    return s[i] - t[i];
}
```

```
void strcpy(char *s, char *t)
{
    int i = 0;
    while((s[i] = t[i]) != '\0')
        i++;
}
```

```
void strcpy(char *s, char *t)
{
    while((*s = *t) != '\0')
    {
        s++; t++;
    }
}
```

```
int strcmp(char *s, char *t)
{
    for( ; *s == *t; s++, t++)
        if(*s == '\0') return 0;
    return *s - *t;
}
```

Notice in the second strcmp and second and third strcpy the use of pointers to iterate through the strings

The conciseness of the last strcmp and strcpy make them hard to understand

```
void strcpy(char *s, char *t)
{
    while((*s++ = *t++) != '\0');
}
```

More On Pointer Arithmetic

- We can also perform subtraction on pointers

```
int a[10] = {...};  
int *ip;  
for(ip = &a[9]; ip >= a; ip--)
```

- Here, we pass to a function the address of the third element of an array (&a[2]) and use pointer subtraction to get to a[0] and a[1])

```
int a[3] = {...};  
printf("%d", addem(&a[2]));
```

```
int addem(int *ip)  
{  
    int temp;  
    temp = *ip + *(ip - 1) + *(ip - 2);  
    return temp;  
}
```

Recall:

a[0] = *a and
a[i] = *(a + i)

If a is an array, and p = &a[0] then we can reference array elements as a[i], *(p+i), but we can also reference them as p[i] and *(a+i) – that is, a and p are both pointers to the array And can be dereferenced by * or by []

Multidimensional Arrays

- As in Java, C allows multidimensional arrays by using more []
 - Example: `int matrix[5][10];`
- Some differences:
 - Because functions can be compiled separately, we must denote all but one dimension of a multiple dimensional array in a function's parameter list
 - `void afunction(int amatrix[ ][10]);`
 - Because arrays are referenced through pointers, there are multiple ways to declare and access 2+ dimensional arrays
 - This will be more relevant when dealing with an array of strings (which is a 2-D array)

```
int a[10][20];  
int *a[10];  
int **a;
```

`*a[4]` –first element of 5th array element
`*a[9]` –first element of 10th array element
`**a` –first element of `a[0]`

```
int *a[3];           // array of 3 pointers  
int x[2] = {1, 2};  
int y[3] = {3, 4, 5};  
int z[4] = {6, 7, 8, 9};  
*a = &x[0];          // a[0] points to x[0]  
*(a+1) = &y[0];       // a[1] points to y[0]  
*(a+2) = &z[0];       // a[2] points to z[0]  
// array a is a jagged array, it is not  
// rectangular, or of equal dimensions
```


Pointers to Pointers

- As indicated in the last slide, we can have an array of arrays which is really an array of pointers or pointers to pointers
 - We may wish to use pointers to pointers outside of arrays as well, although it is more common that pointers to pointers represent array of pointers
 - Consider the following:

```
int a;  
int *p;  
int **q;  
a = 10;  
p = &a;  
q = &p;  
printf("%d", **q);
```

We dereference our pointer p with *p but we dereference our pointer to a pointer q with **q

*q is actually p, so **q is a

// outputs 10

Arrays of Strings Implementation

- We could implement an array of strings as a 2-D array of chars
 - `char array[10][10];`
- This has two disadvantages
 - All strings will be 10 chars long
 - Requires 2 nested for-loops for most operations such as string comparison or string copying, which can become complicated
- Instead, we will implement our array of strings as an array of pointers
 - `char *array[10];`
- Each pointer points to one string
 - Follow the string through the pointer
 - Go to the next string using a for-loop
 - Because `strcpy`, `strcmp`, `strlen` all expect pointers, we can use these by passing an array element (since each array element is a pointer to a string)

Example

```
char *x[ ] = {"hello", "goodbye", "so long", "thanks for all the fish"};
    // our array of strings x is a set of 4 pointers
char *y; // let y be a pointer to a char so it can be used to move through a single string
int i;
for(i=0;i<4;i++)    // iterate for each string in x
{
    y = x[i]; // x[i] is an array, x is really a pointer, so this sets y to x's starting addr.
    while(*y!='\0')    // while the thing y points to is not the end of a string
    {
        printf("%c", *y);    // print what y points to
        y++;    // and go on to the next char in x
    }
    printf("\n");    // separate strings in output with \n
}
```

- Notice that if we had used `char x[ ][ ] = {...}`; then the storage space would have been 4 strings of length 23 (the length of the longest string) or 92 bytes instead of 42 bytes as it is above

Passing Arrays

- When an array is passed to a function, what is being passed is a pointer to the array
 - In the formal parameter list, you can either specify the parameter as an array or a pointer
- Because you can compile functions separately, the compiler must be able to “know” about an array being passed in to a function, so you must specify all (or most) of the definition:
 - The type and all dimensions except for the first

```
int array[100];  
...  
afunction(array);  
...
```

```
void afunction(int *a) {...}  
or  
void afunction(int a[ ]) {...}
```

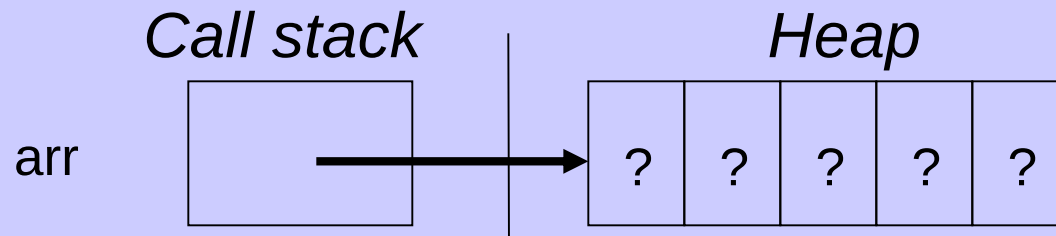
```
int array[5][10][15];  
...  
afunction(array);  
...
```

```
void afunction(int a[ ][10][15]) {...} or  
void afunction(int *a[10][15]) {...} or  
void afunction(int a[5][10][15]) {...} or  
void afunction(int **a[15]) {...} etc
```

Dynamic Allocation of Arrays

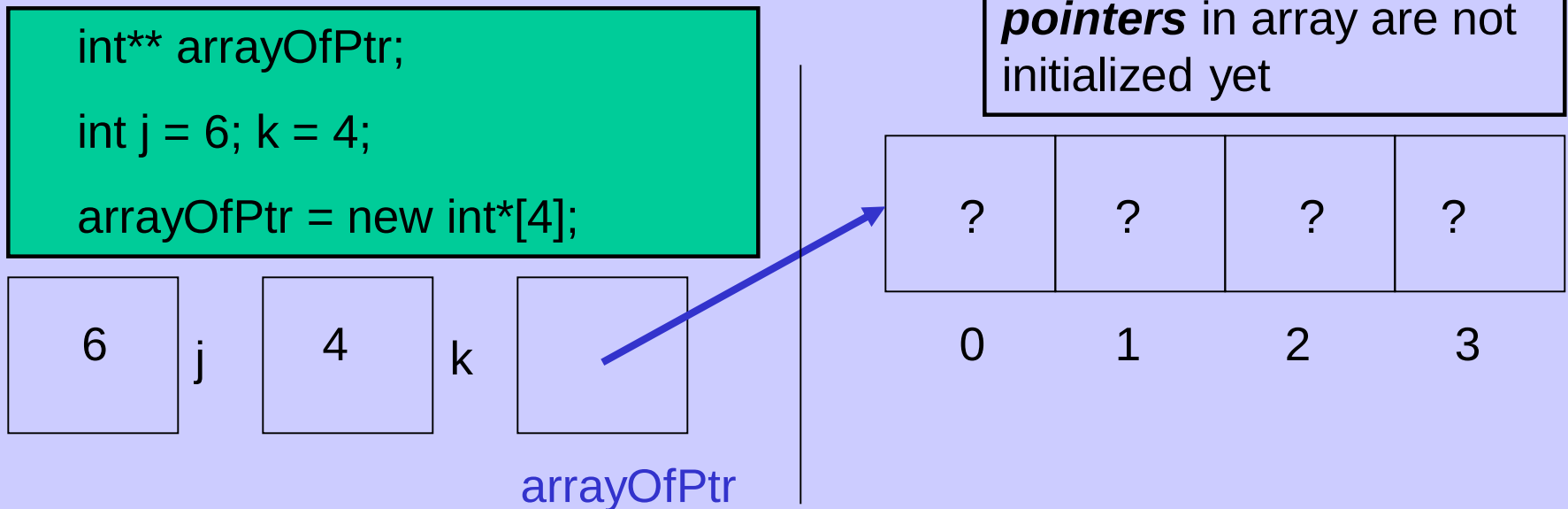
- Use the **new** operator to allocate an array dynamically
- An array name is really a pointer to the array's first element

```
double* arr;  
arr = new double[5];
```



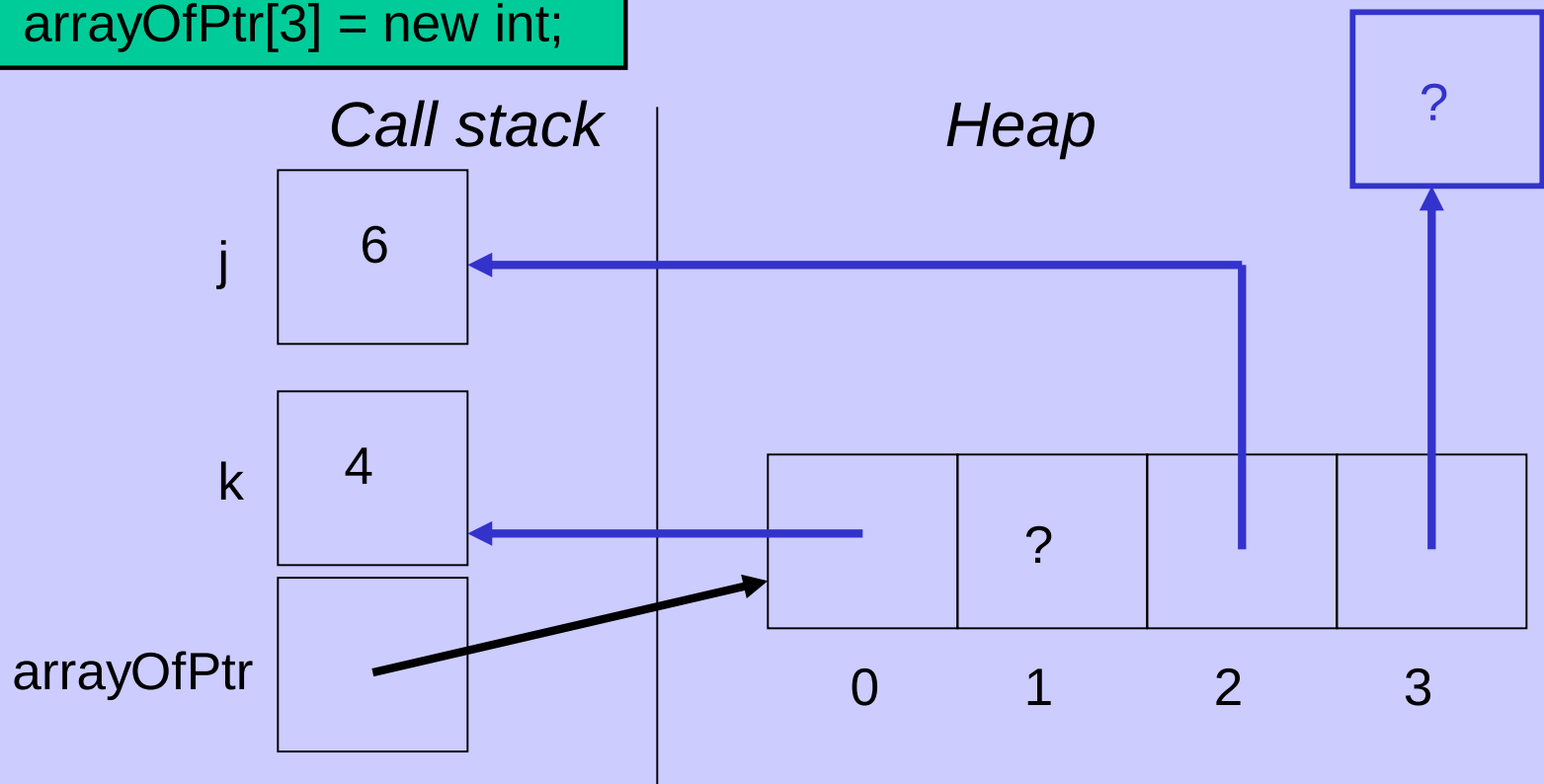
Arrays of Pointers

- It's possible to have arrays of pointers
- The array name is a pointer to an array of pointers:



Arrays of Pointers

```
arrayOfPtr[0] = &k;  
arrayOfPtr[2] = &j;  
arrayOfPtr[3] = new int;
```



Generic Pointers

- Sometimes we need to use pointer variables that aren't associated with a specific data type
- In C++, these pointers simply hold a memory address, and are referred to as ***pointers to void:***

```
void* ptr;
```


Generic Pointers

- Any kind of pointer can be stored in a variable whose type is `void*`
- If we know that a value in a pointer `p` of type `void*` is really of a specific pointer type `x`, and we want to treat the value `p` points to as a value of type `x`, we have to *cast* the pointer to type `x`

Reference Parameters in C++

- Reference parameters just hide the use of pointers from the programmer
- Don't exist in the parent language C, where the programmer must handle *referencing* and *dereferencing* themselves
- C techniques also work in C++; they make explicit use of pointers

Example: Pointer Parameters

- Suppose that a function has the prototype:

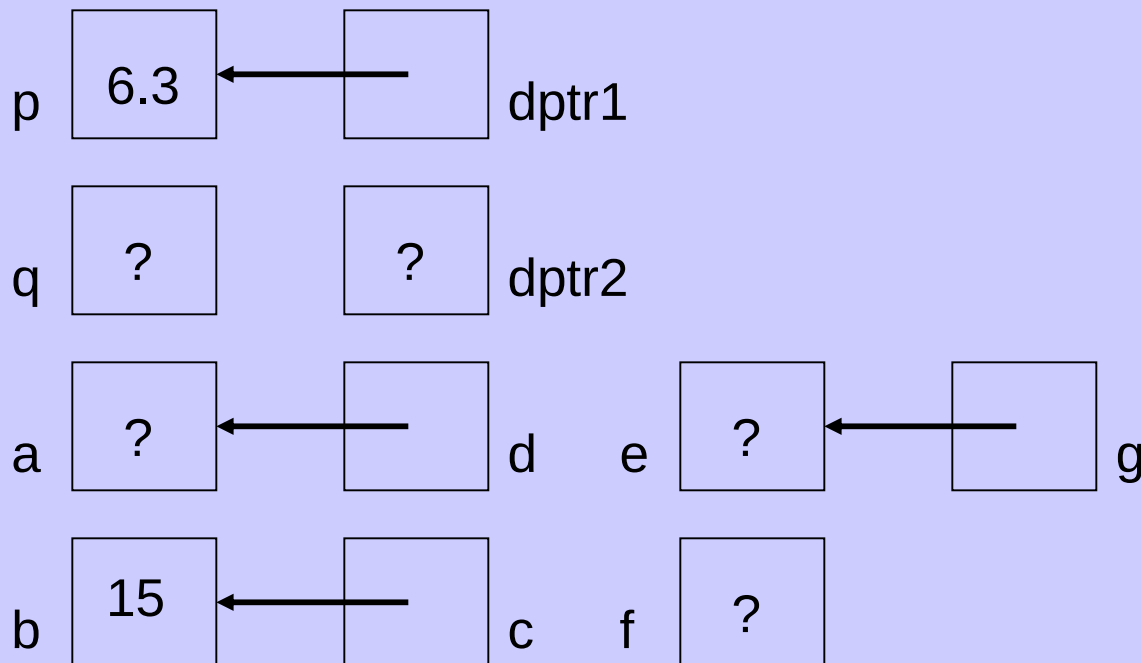
```
void fn( double * x, int ** k)
```

- Inside `fn`, `x` is of type `double*`, and `*x` is of type `double`
- `k` is of type `int**`, `*k` is of type `int*`, and `**k` is of type `int`

Example: Pointer Parameters (cont'd)

- Assume the main program declarations:

```
double p=6.3, q, *dptr1=&p, *dptr2;  
int a, b=15, *c=&b, *d=&a, *e, **f, **g=&e;
```



Example: Pointer Parameters (cont'd)

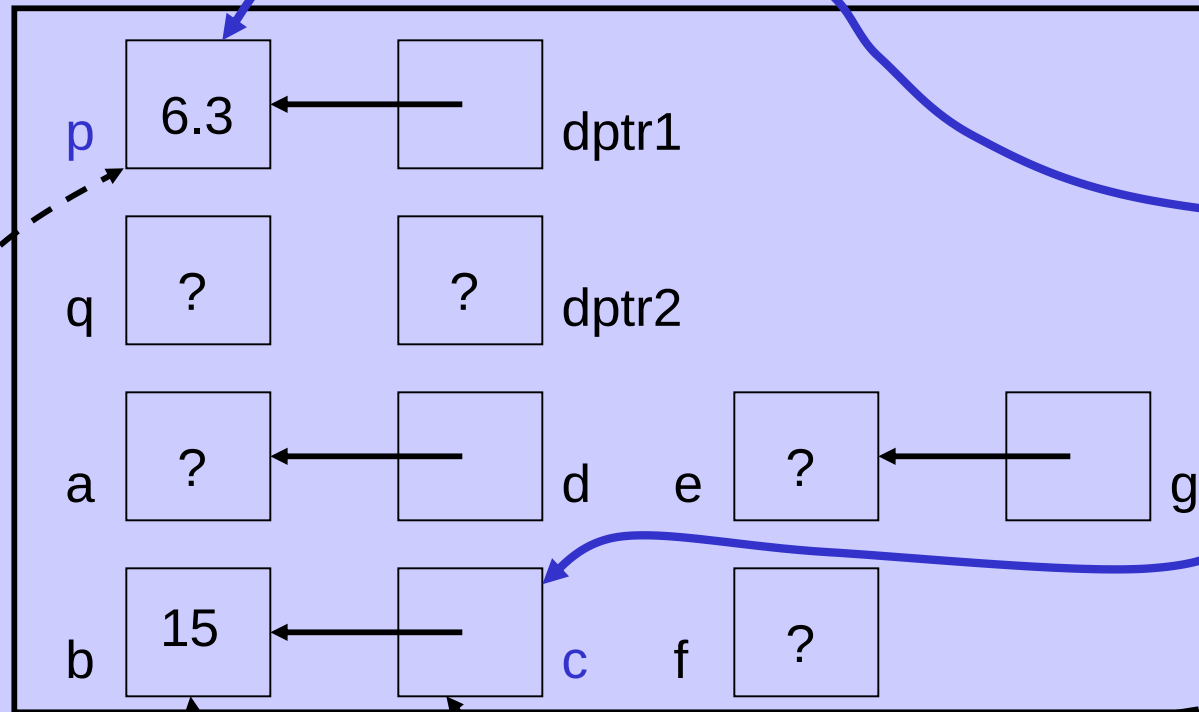
- Each of the following function calls is valid:

```
fn ( &p, &c );
```

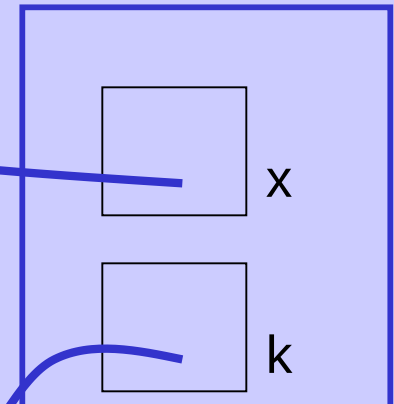
Recall the prototype:

```
void fn( double * x, int ** k)
```

in main



in fn



Recall:

```
int b=15,*c=&b;
```

```
double p=6.3;
```

Example: Pointer Parameters (cont'd)

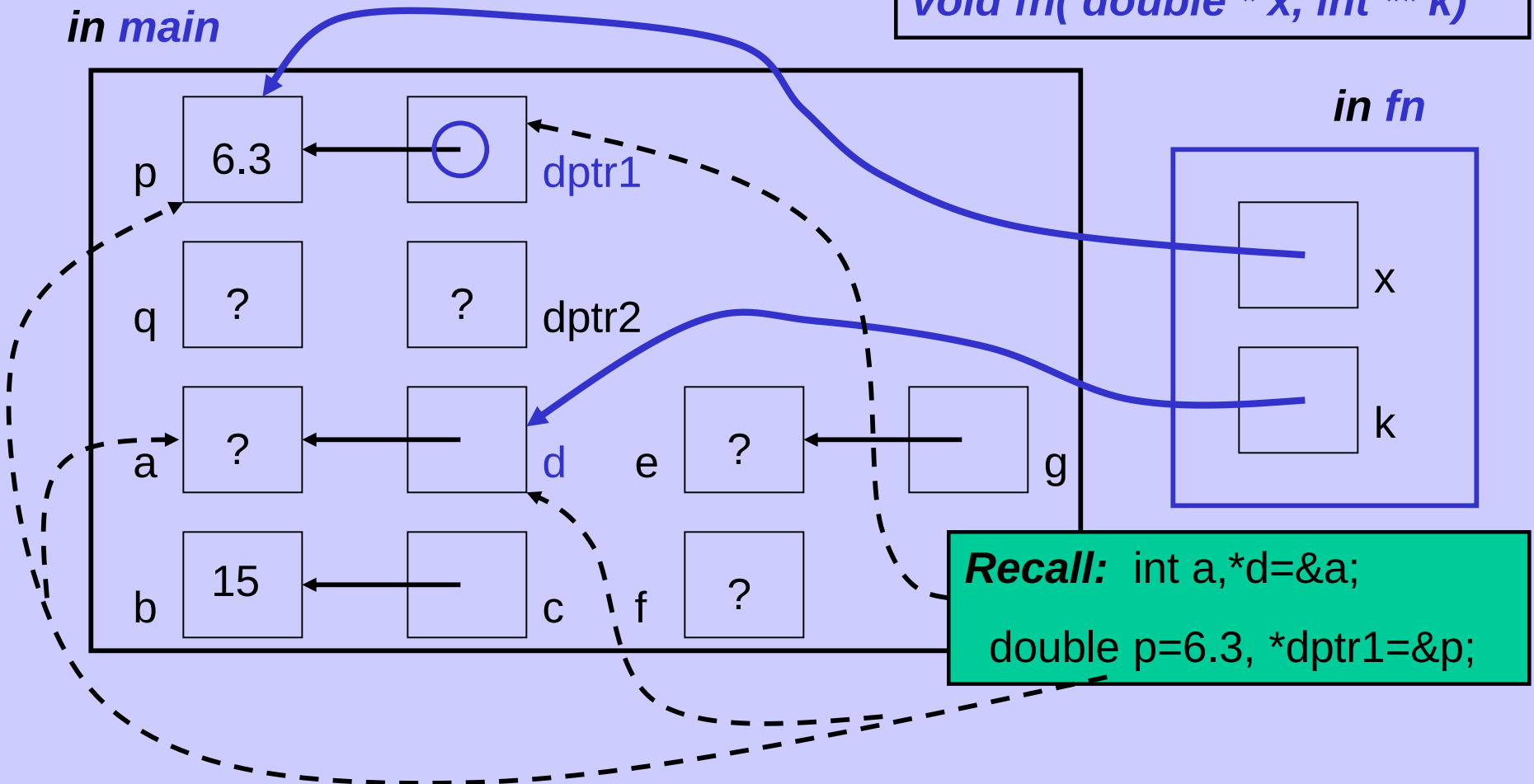
```
fn ( dptr1, &d );
```

Recall the prototype:

```
void fn( double * x, int ** k)
```

in main

in fn



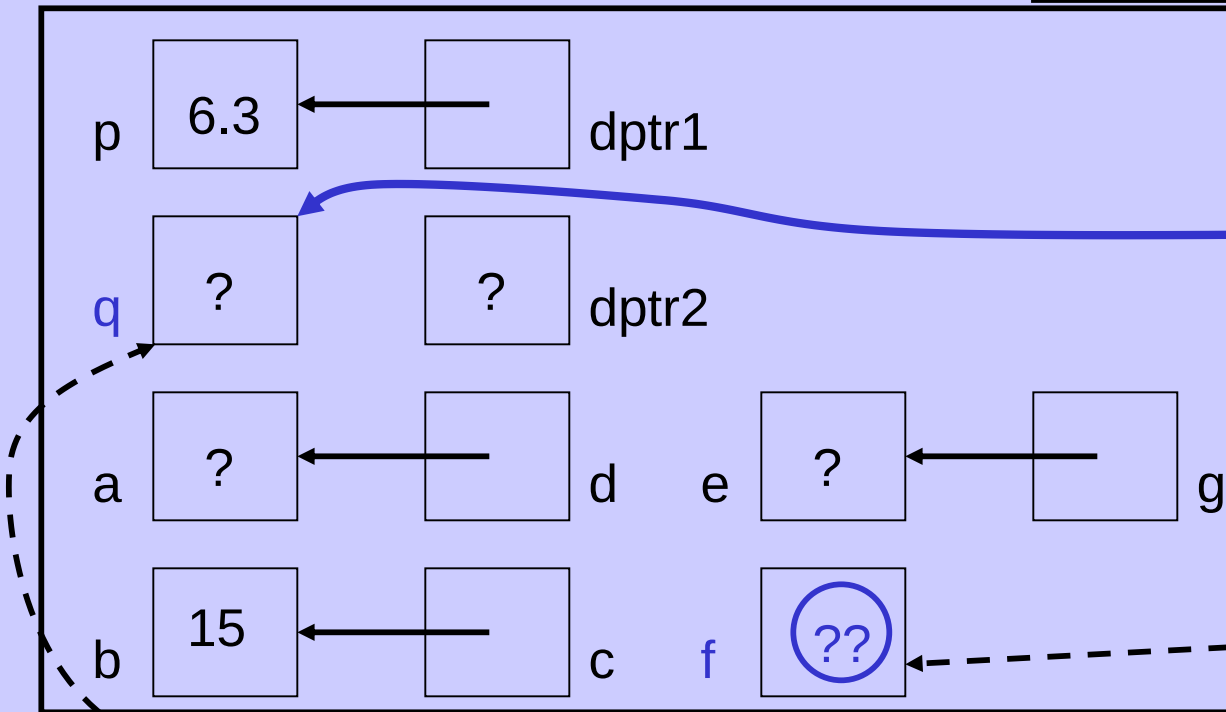
Example: Pointer Parameters (cont'd)

```
fn ( &q, f ); // danger: f doesn't point at anything yet
```

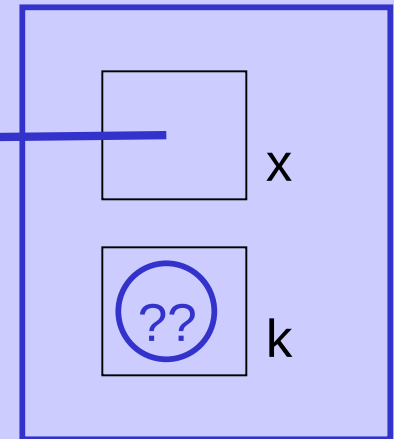
Recall the prototype:

*void fn(double * x, int ** k)*

in main



in fn



Recall: int ** f;
double q;

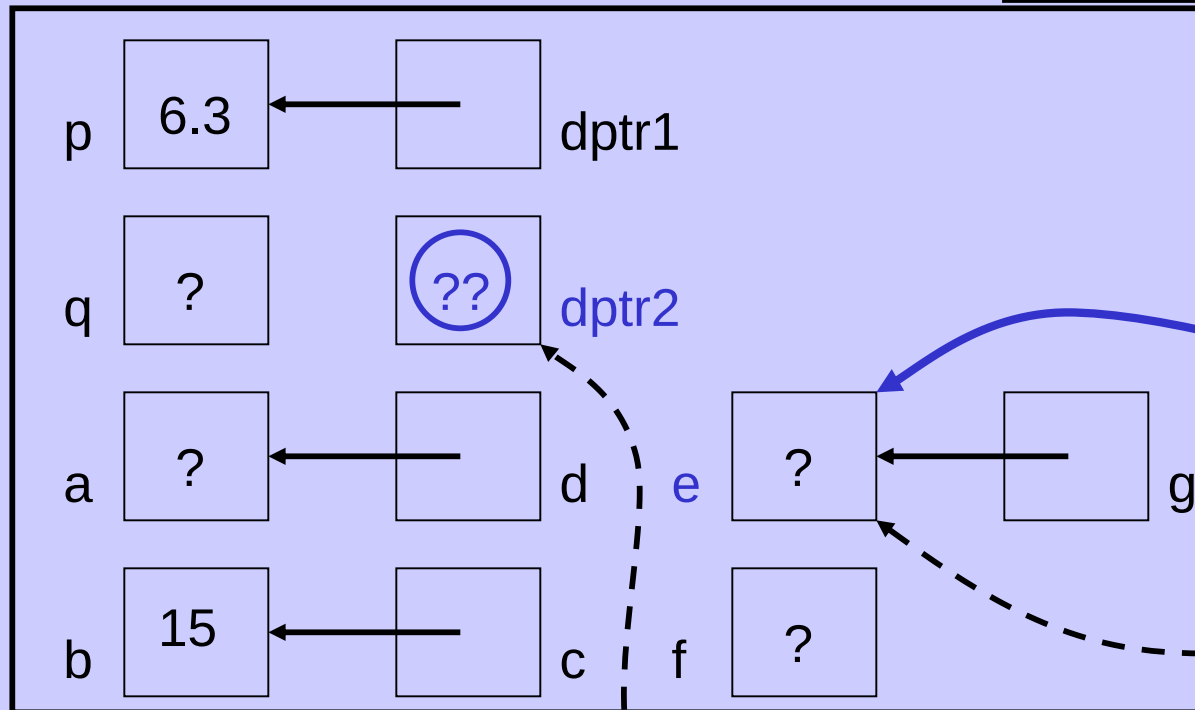
Example: Pointer Parameters (cont'd)

```
fn ( dptr2, &e);  // danger: dptr2 doesn't point at anything yet
```

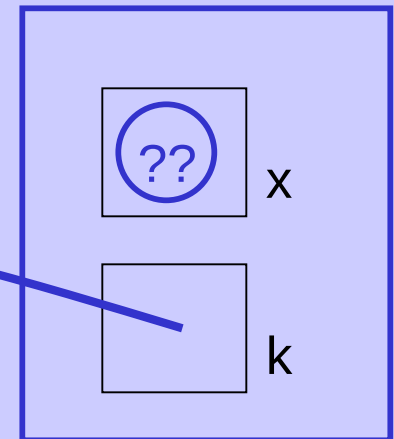
Recall the prototype:

*void fn(double * x, int ** k)*

in main



in fn



Recall: `int * e;`
`double * dptr2;`

Example: Pointer Parameters (cont'd)

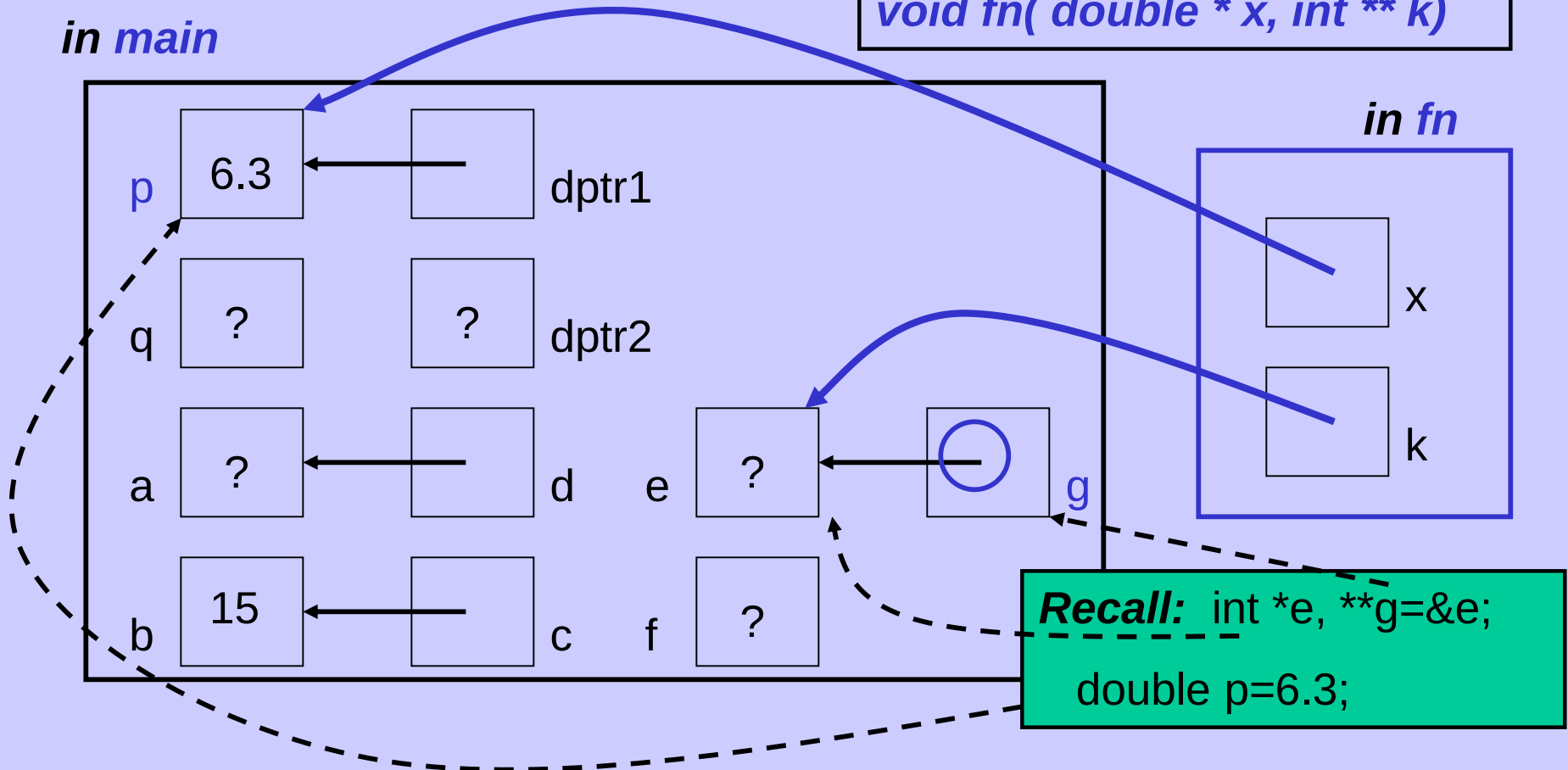
```
fn ( &p, g );
```

in main

Recall the prototype:

*void fn(double * x, int ** k)*

in fn



Example: Pointer Parameters (cont'd)

- The pointer parameters we've shown are all value parameters, so the function works with a copy of the actual parameter, and a call *cannot change* the actual parameter
- The function call *can change* the memory location that the actual parameter references

Why Use Pointer Parameters?

- So that the value being referenced *can be modified* by the function
- *Efficiency*: It takes less time and memory to make a copy of a reference (typically a 4-byte address) than a copy of a complicated object or structure

Arrays as Parameters

- C and C++ do not allow arrays to be copied as parameters
- *Recall:*
 - Array name is really a *pointer* to the first element (location 0)
 - Indexing with square brackets has the effect of dereferencing to a particular array location

Arrays as Parameters

- When we put an array name in a list of actual parameters, we are really passing the pointer to the first element
- Do not use the address operator **&** when passing an array

Arrays as Parameters

- Suppose that we wish to pass an array of integers to a function `fn`, along with the actual number of array locations currently in use
- The following function prototypes are equivalent:

```
void fn( int * arr, int size );
```

```
void fn( int [ ] arr, int size);
```

```
void init1 ( int [ ] a, int s1, int * b, int s2 ) {  
    for (int j = 0; j < s1; j++)  
        a[ j ] = j;  
    for (int k = 0; k < s2; k++)  
        b[ k ] = s2 - k;  
}  
  
int main( ) {  
    int s[4], *t;  
    t = new int [6];  
    t[0] = 6;  
    init1( s, 4, t, t[0] );  
    ...  
}
```

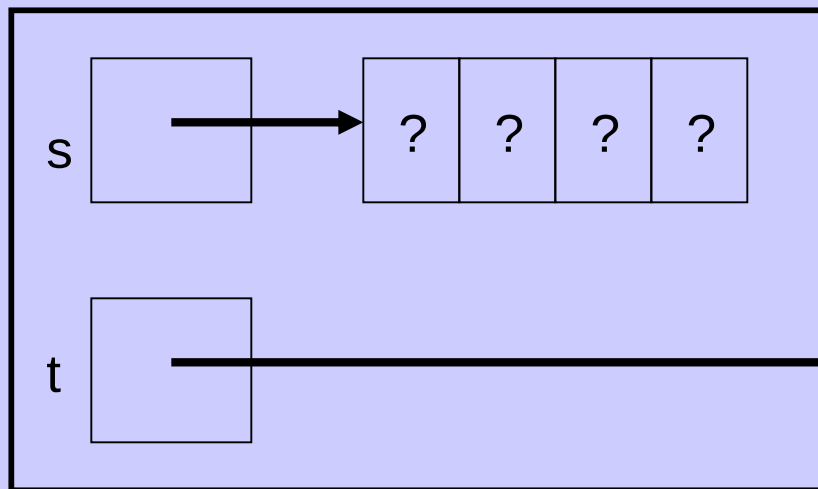
Example

Could use **int *a**
and **int[] b** instead

We trace through
this code on the
next few slides

Example (cont'd)

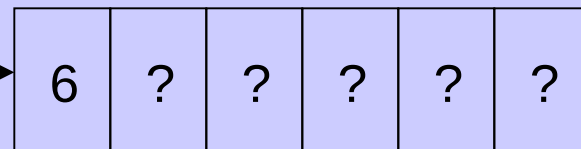
Before the call to `init1`:



Call stack – `main` variables

After execution of:

```
int s[4], *t;  
t = new int [6];  
t[0] = 6;
```

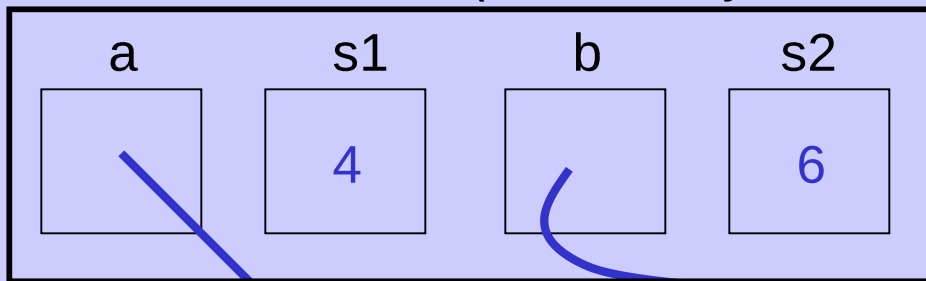


Heap

Example (cont'd)

After call to `init1`, but **before** its execution begins:

Call stack – `init1` (local vars `j` and `k` have been ignored)

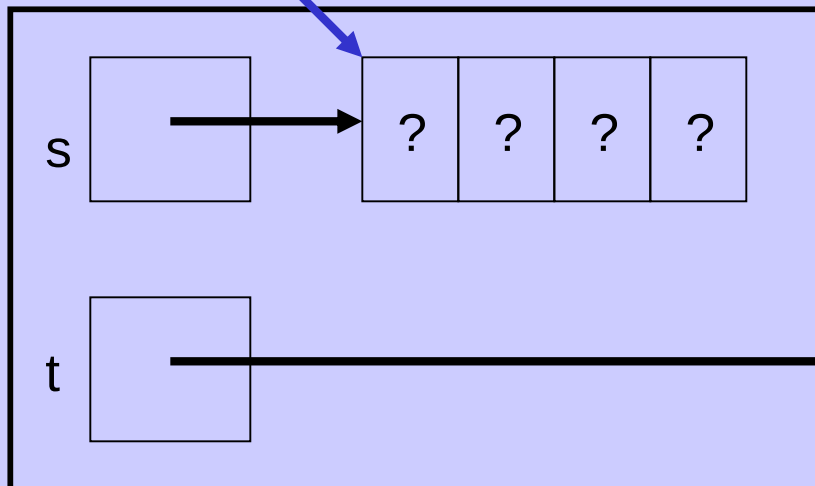


Function prototype:

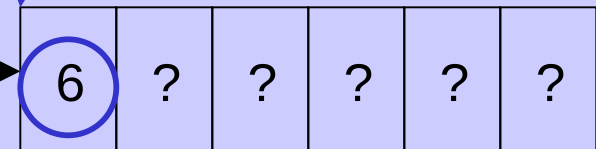
```
void init1 ( int [ ] a, int s1,  
            int * b, int s2 );
```

Function call:

```
init1( s, 4, t, t[0] );
```



Call stack – `main` variables

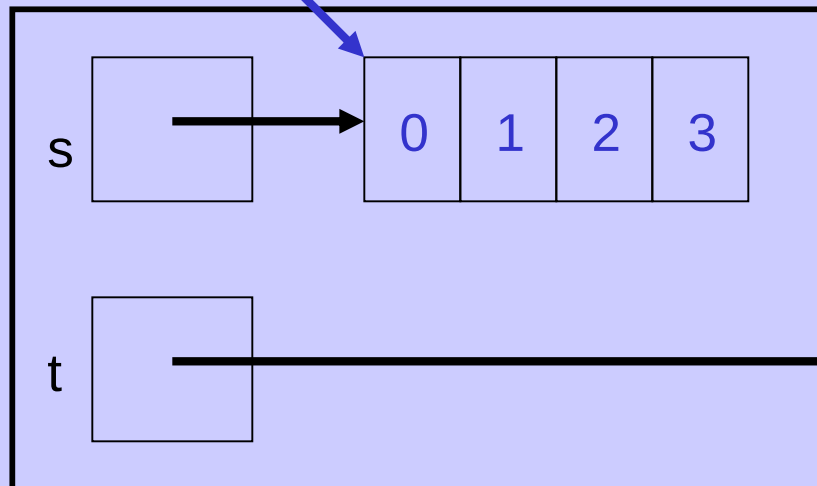
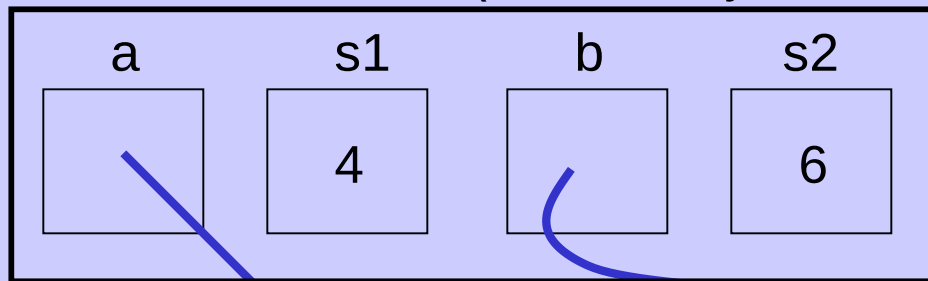


Heap

Example (cont'd)

After execution of `init1`, but **before** return to `main`:

Call stack – `init1` (local vars `j` and `k` have been ignored)



Call stack – `main` variables

After:

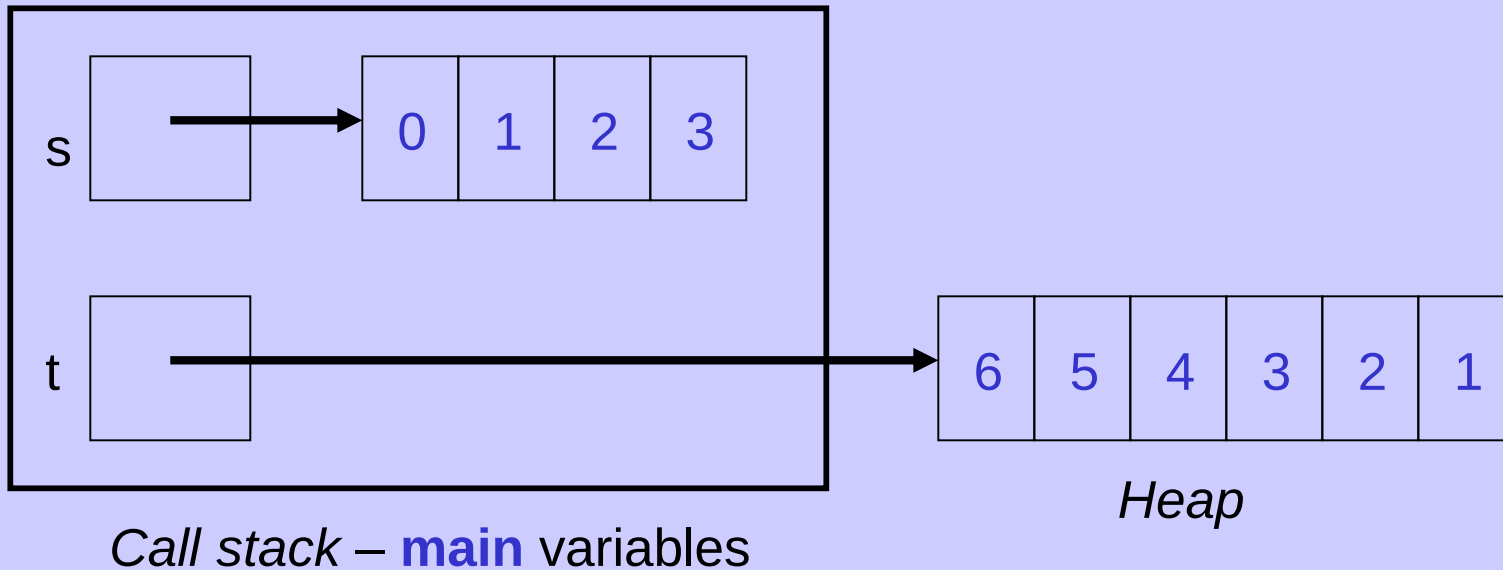
```
for (int j = 0; j < s1; j++)  
    a[j] = j;  
for (int k = 0; k < s2; k++)  
    b[k] = s2 - k;
```



Heap

Example (cont'd)

After returning to `main`:



Portion of the call stack for the call to `init1` has been recycled

Why Is ‘**’ Needed?

- We may want a function to initialize or change a pointer
- Notation can also be used when creating or passing an array of pointers

Using `const` with Parameters

- Use `const` with parameters that aren't pointers to indicate that the parameter will not be altered inside the function
- `const` is used with a pointer parameter to indicate that *the entity being referenced* will not be changed

Using `const` with Parameters

```
void tryConst ( const int k, const int *p ) {  
    k++;           // compiler warning generated  
    (*p) = 83;     // compiler warning generated – attempt to  
                   // change value from the calling module  
                   // that p points at  
    p = new int;  
}
```

No warning from the final line: the value from the calling module that's being referenced is not modified; however, *the function's copy of the value's address* changes